

Notes on the “pirate” dedisperser: the variance map (in progress)

Kendrick Smith

September 7, 2026

Contents

1	The variance map	3
1.1	Setup	3
1.2	Distance function	4
1.3	Tools and current status	4
2	Notes from ch-research campaigns	5
2.1	Campaign 1 setup [human-authored]	5
2.2	Campaign 1 results [agent-authored]	7
2.3	Campaign 2 setup [mostly human-authored]	12
2.4	Campaign 2 results [agent-authored]	15
2.5	Campaign 3 setup [human + agent authored]	17
2.6	Campaign 3 results [agent-authored]	19
2.7	Campaign 4 setup [mostly human authored]	21
2.8	Campaign 4 results [agent-authored]	23
A	Variance maps of a config’s trees are row-restrictions of one another	26
A.1	Inputs	26
A.2	Three consequences of (F1), (F2) and (F5)	27
A.3	Statement 1: early triggers	28
A.4	Statement 2: time downsampling	28
B	Monotonicity of the variance map in the DM bits (no detrender)	29
B.1	Setup	30
B.2	Setting a DM bit spreads the impulse response out	31
B.3	Spreading out decreases the variance	33
B.4	The theorem	33
B.5	Sharpness	34
C	Brute-force computation of the variance map	36
C.1	A as a set of column norms	36
C.2	Undownsamped trees ($\gamma = 0$)	36
C.3	Downsampled trees ($\gamma > 0$)	37
C.4	Placing the one-hot, and sizing N_{time}	37
C.5	Cost, and relation to the other algorithms	39
D	Detrender-free computation of the variance map	40
D.1	Statement of the problem	40
D.2	Variance from autocorrelations	41
D.3	Which delay bits matter	42
D.4	Sparse propagation of the impulse response	42
D.5	Index conventions, and what a group holds	44

D.6	Grouping: entries, the <code>sdbits</code> key, and straddled subbands	44
D.7	The four passes	45
D.8	Per-group SVD, and the lift to Q and W	46
D.9	Truncation threshold	46
D.10	Coarse-graining is a slice, not a maximization	47
D.11	Reducing the rank: global svd-optimization	47
D.12	The other trees are row restrictions of the base tree	48
D.13	Av without the map	48
D.14	Validation	49

1 The variance map

This section is work in progress, and its relation to the rest of the real-time pipeline won't be clear yet!

1.1 Setup

Fix a `DedispersionTree` and use notation from *dedispersion.pdf*, “Tree dedispersion”:

$$r = \text{tree rank} \tag{1.1}$$

$$R = \text{pf.rank} \tag{1.2}$$

$$M = \text{number of multiplets} \tag{1.3}$$

$$P = \text{number of peak-finding profiles} \tag{1.4}$$

$$\gamma = \text{downsampling level (i.e. downsampling factor is } 2^\gamma) \tag{1.5}$$

Consider a linear operator L , whose input is an $(N_{\text{freq}}, N_{\text{time}})$ array and whose output is a $(2^{r-R}, M, P, N_{\text{time}}/2^\gamma)$ array defined by the following sequence of steps:

1. Apply a `GpuDetrenderLps2d` to the $(N_{\text{freq}}, N_{\text{time}})$ array in-place with **no mask**. The parameters of the `GpuDetrenderLps2d` are part of the problem specification – we might “warm up” with a simple case (e.g. $n = W = 0$).
2. Apply tree dedispersion, including peak-finding convolution for each $0 \leq p < P$, but **not** multiplying by a weight array to normalize to “sigmas”, or coarse-graining. Thus, L is the unnormalized dedispersion transform with explicit indices for subbands and peak-finding kernel.

Assume that each element of the input $(N_{\text{freq}}, N_{\text{time}})$ array is an independent mean-zero Gaussian with known variance v_F which depends on frequency channel $0 \leq F < N_{\text{freq}}$ but not time. Then, after dedispersion reaches steady state, the variance of the output array y_{dmp} depends on $0 \leq d < 2^{r-R}$, $0 \leq m < M$, $0 \leq p < P$, but not time. We sometimes “flatten” the indices (d, m, p) to a single index $0 \leq \alpha < 2^{r-R}MP$. Since L is linear, the output variance y_α is linear in the input variance v_F :

$$y_\alpha = \sum_F A_{\alpha F} v_F \tag{1.6}$$

where this equation defines the **variance map** A , a $(2^{r-R}MP)$ -by- N_{freq} matrix.

In its dense matrix representation, the variance map is too expensive to compute (or even store) at CHIME/CHORD scale. This section is concerned with the following central, currently unsolved, problem: can we find a **low-rank** approximation to the variance map $A \approx QW^T$, which can be computed in a reasonable amount of time? That is, we are trying to achieve three goals:

1. We want to minimize the rank of the approximation $A^{\text{approx}} \equiv QW^T$.
2. We want the approximation $A^{\text{approx}} \approx A^{\text{true}}$ to be accurate, in the sense that a certain distance function $D(A^{\text{true}}, A^{\text{approx}})$ is small. The choice of distance function is unusual, and is discussed in the next subsection.
3. We want A^{approx} to be computable in a reasonable amount of time. In particular, we'd like to avoid computing every matrix element of A^{true} , since this is cost-prohibitive at CHIME/CHORD scale. This aspect of the problem is particularly challenging, so I suggest “warming up” by exploring subscale examples, where computing A^{true} is feasible, and finding a scalable solution later.

1.2 Distance function

Given matrices $A_{\alpha F}^{\text{true}}, A_{\alpha F}^{\text{approx}}$, our distance function $D(A^{\text{true}}, A^{\text{approx}})$ is defined as follows. First, we define vectors $y_{\alpha}^{\text{true}}, y_{\alpha}^{\text{approx}}$ by:

$$y_{\alpha}^{\text{true}} \equiv \sum_F A_{\alpha F}^{\text{true}} \quad y_{\alpha}^{\text{approx}} \equiv \sum_F A_{\alpha F}^{\text{approx}} \quad (1.7)$$

Then we define:

$$D(A^{\text{true}}, A^{\text{approx}}) \equiv \begin{cases} \infty & \text{if } A_{\alpha F}^{\text{approx}} < A_{\alpha F}^{\text{true}} \text{ for any } (\alpha, F) \\ N_{\alpha}^{-1} \sum_{\alpha} f(y_{\alpha}^{\text{approx}}/y_{\alpha}^{\text{true}}) & \text{otherwise} \end{cases} \quad (1.8)$$

where $N_{\alpha} = \sum_F 1$ is the number of α -values, and $f(x)$ is defined by:

$$f(x) \equiv \frac{x - 1}{1 + (x/10)} \quad (1.9)$$

This is implemented by `VarianceMap.get_distance()` (in `pirate_frb/varmap/`), which should be used wherever a distance is reported, so that results are comparable. Its companion `VarianceMap.measure_admissibility()` reports the worst per-element ratio $\max_{\alpha F} (A_{\alpha F}^{\text{true}}/A_{\alpha F}^{\text{approx}})$ (`max_r`) together with the (α, F) where it occurs, and with `inflate=True` also the distance `D_inflated` of A^{approx} rescaled by that ratio. These are more informative than D alone: if the max ratio is only a little larger than 1, then inflating A^{approx} slightly gives a finite distance, so such an approximation is nearly usable – whereas $D = \infty$ by itself does not distinguish it from a hopeless one.

This unusual distance function has the following properties:

1. Underestimating any matrix element of A is catastrophic – the approximation is considered infinitely bad.
2. Overestimating is okay, but our goal is to minimize the average level of overestimation.
3. Extreme overestimation is fine, as long as it's confined to a small number of α -values. (If we send $y_{\alpha} \rightarrow \infty$ for a few α values, then it has a bounded effect on the distance function.)

The motivation is as follows. In the real-time search, we normalize dedispersion outputs to “sigmas” by multiplying by $(Av)^{-1/2}$, where v_F is a real-time estimate for the RFI-masked variance. We search a gigantic number of trials ($\sim 10^{15}$) for rare events, and we use a high threshold ($9\text{--}10\sigma$) due to the high trial factor.

Underestimating variance is catastrophic, even in corner cases, since it can generate a large number of false positives in the far tail of the SNR distribution. The RFI mask is unpredictable and can produce corner cases, such as a small number of frequency channels contributing. Therefore, we impose a strict constraint (per matrix element!) that we never underestimate variance.

In contrast, overestimating variance is undesirable, since it reduces the total event rate, but this is a statistical concern, not a hard constraint. Because it's a statistical concern, our cost function focuses on the typical case where all channels have similar variance, not per-matrix-element analysis.

1.3 Tools and current status

- The `pirate_frb.varmap` subpackage contains tools for working with variance maps. The central object is a `VarianceMap` class which represents a variance map A , in a flexible sense: A can be either exact or approximate, can be either dense or low-rank, and can be coarse-grained (in the sense defined by Eq. (1.10) below). The `VarianceMap` class has methods that can manipulate a variance map (e.g. saving to an asdf file).

- There is a “brute-force” algorithm to compute the variance map A in dense form (§C), for any linear operator L which commutes with time translations. In our case of interest, where L is defined by a `DetrenderLps2dParams` and a `DedispersionConfig`, this algorithm is implemented (CLI: `pirate_frb varmap bf`). This may be a useful “crutch” for initial exploration, although in the long run it won’t scale to CHIME/CHORD. It would be a significant milestone to develop an algorithm which works well starting from a dense representation of A .
- In the “detrender-free” special case, there is a theorem that A does not have full rank (in general), and an algorithm for computing A in low-rank form, without computing a dense matrix as an intermediate. This algorithm is implemented as `pirate_frb varmap df`. Unfortunately, this algorithm does not generalize to the case where a `GpuDetrenderLps2d` is specified.
- There is a general notion of coarse-graining the variance map $A_{\alpha F}$. Recall that $0 \leq \alpha < (2^{r-R}MP)$ is a “flattened” index. Let $0 \leq \beta < (2^{r-L}NP)$ be a coarser “flattened” index obtained by coarse-graining DM. Here, $L \geq R$ is a coarse-graining factor, and $N \leq M$ is the number of frequency subbands (a multiplet is a subband plus a fine-grained DM). Define the coarse-grained variance map $\bar{A}$ by:

$$\bar{A}_{\beta F} = \max_{\alpha \in \beta} A_{\alpha F} \quad (1.10)$$

Conversely, $\bar{A}$ (or a low-rank approximation to $\bar{A}$) can be “lifted” to an approximation of A . At CHIME/CHORD scale, and with $L = 6$ (say), working with $\bar{A}_{\beta F}$ is much more feasible than working with A , and the distance penalty due to the “lift” turns out to be small. The `varmap bf` and `varmap df` CLI commands have a `-L` flag to output a coarse-grained variance map.

- One detail: when we either coarse-grain a map $A_{\alpha F} \rightarrow \bar{A}_{\beta F}$, or construct a low-rank approximation $A^{\text{approx}} \approx A^{\text{true}}$, we retain the *true, fine-grained* vector y_{α}^{true} defined in Eq. (1.7). We also retain a boolean flag `is_admissible` which keeps track of whether the constraint $A_{\alpha F}^{\text{approx}} \geq A_{\alpha F}^{\text{true}}$ is satisfied. Together, these two pieces of data suffice to compute $D(A^{\text{true}}, A^{\text{approx}})$, and thereby gauge the quality of the approximation (including coarse-graining distance penalty).
- In the general case (i.e. including a `GpuDetrenderLps2d`), we’re currently in the process of exploring algorithms which start from the “brute-force” dense A , and obtain a low-rank approximation, using a sequence of steps (e.g. coarse-graining, SVD decomposition, linear programming). Since this is work in progress and the details are still evolving, I haven’t written up the details in the tex – see `misc/varmap_ladder/README.md` for current status.

2 Notes from ch-research campaigns

In this section, we’ll explore algorithms for finding low-rank approximations, under the assumption that the dense A -matrix has been precomputed and saved to disk.

This section is organized as alternating subsections which are human-authored or agent-authored. The human-authored sections usually propose ideas for the agent to explore, and the agent-authored sections usually report results (after running a “campaign” via the `ch-research` skill).

2.1 Campaign 1 setup [human-authored]

Let’s start by considering the following question. Suppose that we’ve precomputed the variance map A , given a `DedispersionConfig` and a `DetrenderLps2dParams`, and written it to disk. What are some interesting algorithms for finding a low-rank approximation with small $D(A^{\text{true}}, A^{\text{approx}})$?

An ideal milestone would be to have some code which reads a precomputed variance map A , and returns a few (matrix rank, distance) pairs, to get a sense for how the level of approximation might depend on the rank of the approximation.

Some practical notes, to get started:

- To compute an A -matrix, use `pirate_frb varmap bf <config> -o A.asdf` (the brute-force sweep, §C, which takes an optional `<detrender>` positional argument or `--no-detrender`) or `pirate_frb varmap df <config>` (the analytic map, no detrender). To read one back, use `varmap.VarianceMultiMap.from_asdf()`, and to check one against Monte Carlo, `pirate_frb varmap mc A.asdf`.
- To evaluate the distance function, use `VarianceMap.get_distance()` (in `pirate_frb/varmap/`, see §1.2). Please use it, rather than reimplementing D , so that distances reported by different experiments are comparable. Besides D , report `max_r`, `max_diff` and `D_inflated` from `measure_admissibility(inflate=True)`: the first and third distinguish an approximation which is nearly usable from one which is hopeless, and the second is the accuracy figure (`max_r` is an inflation factor, not an error – see §1.2).
- There is no subscale config in `configs/dedispersion/` yet, so part of the task is to create a variety of them, exploring the `DedispersionConfig` and `DetrenderLps2dParams` parameter space. We are more interested in *typical* cases than in adversarial or edge cases: the goal is an algorithm that works well on the configs we would actually run, not one that survives a worst case we will never encounter.

Here are some initial thoughts and directions that may be interesting, but I encourage you to propose new ideas and directions:

- I suspect that heuristic algorithms and empirical approaches will work best, and that making rigorous guarantees will be hard. (But I’d love to be wrong about this!)
- Let’s disable downsampled trees and early triggers. I suspect that if we can solve the problem for a single `DedispersionTree`, we’ll be able to solve the multi-tree case.
- To start out, let’s also disable frequency subbands, so that the main parameters to explore are the frequency zones, and the `GpuDetrenderLps2d`. Let’s also use subscale instances (say rank-10 and ~ 400 frequency channels). Later, we can scale up, and add frequency subbands.
- Our distance function (1.8) is discontinuous, which makes it hard to use optimization-based approaches. It may be useful to “relax” to a continuous distance function in an intermediate stage. Or, if continuity turns out to be very helpful, then we could think about changing the definition of the cost function.
- Suppose we have an accurate approximation $A_1 = QW^T$ with a large rank (say 1000). Can we find an algorithm to search for a lower-rank (say 100) representation A_2 with small $D(A_1, A_2)$? If such an algorithm exists, it could be useful for “bootstrapping” low-rank approximations from medium-rank ones.
- Is coarse-graining DM a useful idea? Recall that $0 \leq \alpha < (2^{r-R}MP)$ is a “flattened” index. Let $0 \leq \beta < (2^{r-L}NP)$ be a coarser “flattened” index obtained by coarse-graining DM. Here, $L \geq R$ is a coarse-graining factor, and $N \leq M$ is the number of frequency subbands (a multiplet is a subband plus a fine-grained DM). Define the coarse-grained variance map $\bar{A}$ by:

$$\bar{A}_{\beta F} = \max_{\alpha \in \beta} A_{\alpha F} \quad (2.1)$$

One approach to controlling rank/cost is to find a low-rank approximation to $\bar{A}$, and “lift” it to a low-rank approximation to A . (Note that our current “production” GPU kernels use coarse-grained weights, which implicitly assumes that this approach is viable at some level.)

- If we’re having trouble at low DM specifically, then we could inflate variance estimates at low DM (with little or no change in the rank). Given our unusual distance function D , it may not change the value of D dramatically.

2.2 Campaign 1 results [agent-authored]

This subsection describes an algorithm which came out of an initial exploration of the milestone above, and the numbers it produces. The short version: the approximation which works is a *one-sided nonnegative factorization at coarse-grained DM*, built by alternating two linear programs. It is more accurate than any clustering we tried, it turns out to be cheaper to apply than the alternatives, it never forms the dense A , and its output is the array the production peak-finding kernel already stores.

Throughout, Q and W are the factors of $A^{\text{approx}} = QW^T$ as in §1.1: W is N_{freq} -by- K and Q is $(2^{r-R}MP)$ -by- K . Note the symbol clash with the `GpuDetrenderLps2d` time half-width W of *detrending.pdf*, “2-d detrending”; below, the detrender’s half-width is always named in words, never as W .

Why not least squares. It is worth being explicit about why the obvious tool fails, because it shapes everything else. The constraint in Eq. (1.8) is *per matrix element*: a single $A_{\alpha F}^{\text{approx}} < A_{\alpha F}^{\text{true}}$ anywhere sends D to infinity. So this is a *one-sided* (upper-bound) approximation problem, not a least-squares problem, and a truncated SVD is not merely suboptimal but usually inadmissible. A truncation has negative entries; clipping them to zero still leaves it underestimating A somewhere, and no rescaling can repair that. On a rank-8 map with a detrender, truncations at rank 8, 16, 32 and 64 are all inadmissible ($D = \infty$); the first admissible one is rank 128, and it scores $D = 0.10$ where a rank-128 envelope scores 0.054. Least-squares accuracy is close to irrelevant here, and at low rank it is actively misleading.

The exception is instructive. With *no* `GpuDetrenderLps2d`, A really is close to low rank – numerically $\text{rank}(A) = 54$ to 138 on our maps – and a clipped, inflated rank-16 SVD reaches $D \approx 10^{-6}$, four orders of magnitude better than any clustering at the same rank. Frequency subbands alone do not break this. It is the `GpuDetrenderLps2d` that destroys the algebraic structure (*detrending.pdf*, “2-d detrending”: it smears every one-hot across its whole spline zone, which also destroys the exact zeros of A), and once it is switched on the SVD is dead at every rank below N_{freq} . The algorithm below is aimed at that case, since it is the one we run.

The shape of the approximation: a coarse-grained Q . Partition the $N_\alpha \equiv 2^{r-R}MP$ output indices α into N_β groups, indexed by β , and write $\beta(\alpha)$ for the group containing α . Give each *group* a single coefficient vector $q_\beta \in \mathbb{R}_{\geq 0}^K$, and let every α in that group share it:

$$A_{\alpha F}^{\text{approx}} = (W q_{\beta(\alpha)})_F \quad (2.2)$$

To fix the shapes: W is N_{freq} -by- K , so its K columns are frequency vectors; q_β is a column vector of K nonnegative numbers; $W q_\beta$ is therefore an N_{freq} -vector, and $(\cdot)_F$ takes its F -th component. In words, Eq. (2.2) says *every output row in a group is the same nonnegative combination of the same K columns of W* .

This is the notes’ own coarse-graining of §2 with one generalization: instead of giving each group a single column of W , we let it use a nonnegative *combination* of them.

Relation to Q , and the degrees of freedom. Collect the q_β as the rows of a N_β -by- K matrix $\bar{Q}$. Then the Q of $A^{\text{approx}} = QW^T$ is $\bar{Q}$ with its rows duplicated across each group,

$$Q = S\bar{Q}, \quad S_{\alpha\beta} = \begin{cases} 1 & \text{if } \beta(\alpha) = \beta \\ 0 & \text{otherwise} \end{cases} \quad (2.3)$$

where S is the N_α -by- N_β one-hot group-membership matrix. So Q formally has N_α rows, but only N_β *distinct* ones, and the free parameters are W and $\bar{Q}$ alone: $K(N_{\text{freq}} + N_\beta)$ of them, against $K(N_{\text{freq}} + N_\alpha)$ for an unrestricted per-row factorization. The rank is K either way.

That row duplication is not a technicality – it is where the practical win comes from, since $N_\alpha/N_\beta = 2^{L-R}(M/N)$ is large (233 at CHORD tree 0). At the CHIME tree-0 geometry below, with $N_\alpha = 524288$, $N_\beta = 8192$, $N_{\text{freq}} = 16384$ and $K = 32$: W carries 5.2×10^5 parameters and $\bar{Q}$ carries 2.6×10^5 dense – or

1.6×10^4 nonzeros once q_β is capped at two nonzero entries – against 1.7×10^7 for a per-row Q of the same rank.

The groups are exactly the β of §2, so that $N_\beta = 2^{r-L}NP$: coarse-grain DM by a factor 2^{L-R} , and never merge across a frequency subband or a peak-finding profile. An ablation over the three axes says this is not a stylistic choice – merging subbands costs a factor 489 to 3174 at equal rank, merging profiles a factor 11 to 175, whereas merging *fine-grained* DM within a multiplet is nearly free (a factor 1.0 to 3.9). The reason subbands cannot be merged is that they have disjoint frequency support, and this survives the detrender: the support stops being disjoint but stays concentrated, which is enough.

Define the coarse-grained variance map, as in §2,

$$\bar{A}_{\beta F} = \max_{\alpha \in \beta} A_{\alpha F} \quad (2.4)$$

Then any q_β satisfying $Wq_\beta \geq \bar{A}_{\beta,:}$ elementwise is automatically admissible for every fine row in the group, since $(Wq_\beta)_F \geq \bar{A}_{\beta F} \geq A_{\alpha F}$. **This is the pivot of the whole algorithm:** admissibility becomes a property of $\bar{A}$ alone, so the optimization never needs to see a dense row of A .

What the algorithm needs, and why it is cheap. Only three things: $\bar{A}$, the fine row sums $y_\alpha = \sum_F A_{\alpha F}$, and the group labels (pure geometry). The row sums are needed only to *score* the result, not to build it. Both $\bar{A}$ and y are computable in a single streaming pass, because the brute-force sweep of §C already produces A one input channel (one column) at a time: max-reduce each column into its groups as it appears, accumulate y , and discard the column. For a CHIME-shaped tree this is 1.0 GiB of $\bar{A}$ instead of 64 GiB of A ; for CHORD, 5.4 GiB instead of 1251 GiB. Measured against a dense reduction on the 64 GiB case, the streamed $\bar{A}$ is bit-identical, uses 1.9 GiB of peak RSS against 128.5 GiB, and is 20% faster.

Step 1: constructing W (this is also the initialization). We need an initial W , i.e. K frequency vectors from which every group’s row can be covered. Take the N_β rows of $\bar{A}$, normalize each to unit sum,

$$\bar{u}_{\beta F} = \bar{A}_{\beta F} / \sum_{F'} \bar{A}_{\beta F'} \quad (2.5)$$

and cluster them by greedy agglomerative merging: start with one cluster per group, and repeatedly merge the pair whose merge increases the objective least. The column of W for cluster c is the elementwise max over its members, $W_{Fc} = \max_{\beta \in c} \bar{u}_{\beta F}$.

Two things about this are worth understanding rather than just implementing.

Why shapes and not rows. If a group is given a single column of W , it is always free to rescale it – $q_\beta = se_c$ costs nothing in rank – and the optimal s is $\max_F \bar{A}_{\beta F} / W_{Fc}$. Substituting it, the resulting $y_\beta^{\text{approx}} / y_\beta^{\text{true}}$ is invariant under $W \rightarrow cW$ and does not involve the row’s overall normalization at all. So *once rescaling is allowed, only the shapes matter*, and clustering the raw rows optimizes the wrong thing – visibly so, since it produces a non-monotone frontier in K . Clustering the unit-sum shapes instead is worth 3–20% (typically 11–14%) at no extra cost, and the argument is stronger for the full algorithm below, whose q_β is exactly scale-invariant in each atom. Note this construction of W needs only $\bar{A}$ – not even y .

Why this is a good enough starting point. The initial W is a max envelope, so the one-hot $q_\beta = e_{c(\beta)}$ is feasible: the alternation starts from an admissible point with a known finite D , and every step below preserves feasibility. The W also does not have to be optimal, only reasonable: building it from $\bar{A}$ at the same coarse-graining used for Q is within 3.6% of building it from a finer $\bar{A}$, and within 3.5% of building it from the dense A . So there is no reason to keep a finer $\bar{A}$ around – the memory footprint is set by Q alone.

Step 2: the Q-step (optimize Q , holding W fixed). Let $s_c = \sum_F W_{Fc}$, so that a group’s approximate row sum is $s \cdot q_\beta$. Since f in Eq. (1.8) is strictly increasing, minimizing D over q_β is exactly minimizing the row sum, and the groups are independent. So the Q-step is, for each β separately, the covering linear

program

$$\min_{q_\beta \geq 0} s \cdot q_\beta \quad \text{subject to} \quad Wq_\beta \geq \bar{A}_{\beta,:} \quad (2.6)$$

with K variables and N_{freq} constraints. Three remarks. It is *exact*, not a heuristic – given W , no better Q exists. It uses only that f is increasing, so it is insensitive to the exact shape of the cost function. And it is embarrassingly parallel over groups. At $N_{\text{freq}} = 16384$ it should be solved with constraint generation (start from a subset of channels, add violated ones, repeat); this is $8.7\times$ faster than handing the solver all the constraints and agrees with it to 10^{-9} , and without it the method looks unaffordable when it is not.

Step 3: the W-step (optimize W , holding Q fixed). This one needs a small trick, because the objective depends on W through the column sums while the constraint depends on W elementwise, so it does not decouple as written. Observe that f is concave:

$$f'(x) = \frac{1.1}{(1+x/10)^2} > 0, \quad f''(x) < 0 \quad (2.7)$$

For a concave f the tangent at the current iterate is a global *upper* bound, $f(x) \leq f(x_0) + f'(x_0)(x - x_0)$, so minimizing the tangent is a majorize-minimize step and is guaranteed not to increase the true objective. The majorizer is linear in y , hence linear in W : with weights $w_\alpha = f'(x_\alpha)/y_\alpha^{\text{true}}$ evaluated at the current iterate, it is $\sum_{F,c} W_{Fc} (\sum_\alpha w_\alpha Q_{\alpha c})$, which *decouples over frequency channels*. So the W-step is, for each channel F separately,

$$\min_{W_{F,:} \geq 0} g \cdot W_{F,:} \quad \text{subject to} \quad QW_{F,:} \geq \bar{A}_{:,F}, \quad g_c \equiv \sum_\alpha w_\alpha Q_{\alpha c} \quad (2.8)$$

with K variables and N_β constraints. Again only $\bar{A}$ is touched.

Iteration schedule. Feasibility is an invariant rather than a hope: after a W-step the current q_β is still feasible for the next Q-step, and after a Q-step the current $W_{F,:}$ is still feasible for the next W-step. Starting from the admissible envelope of Step 1, every LP in every run we did was feasible.

In practice, **run three Q-steps and two W-steps and stop**. The first Q-step is 68–94% of the total gain; the W-step is worth only a further 7–18%, and *nothing at all* on a map with no detrender. Monotonicity was checked over 435 consecutive half-steps: the only increases were $\sim 10^{-5}$ relative, all attributable to the admissibility repair below.

Numerical care (this is not optional). An LP solution that is “feasible” to the solver can still violate $Wq \geq \bar{A}$ enough to make D infinite. The tolerance to budget for is *relative*, but a solver’s tolerance is *absolute* – HiGHS’s default primal feasibility tolerance is $\sim 10^{-7}$, which on a matrix element of 10^{-12} is meaningless. On one subbanded map the W-step set $W_{Fc} = 0$ where $A^{\text{true}} > 0$, which no rescaling can repair. Two fixes, both cheap:

1. **Equilibrate every constraint to a right-hand side of 1**, i.e. rewrite $Mx \geq b$ as $(M/b)x \geq 1$, dropping the $b = 0$ rows (automatically satisfied since $M, x \geq 0$). The solver’s absolute tolerance is then a uniform relative one. This brought the worst violation down to 1.3×10^{-7} – exactly the solver tolerance – on every map.
2. **Repair exactly and locally afterwards**, rather than applying a blanket multiplicative margin: scale row β of Q by $\max(1, \max_F \bar{A}_{\beta F} / (Wq_\beta)_F)$, and similarly per channel after a W-step. Scaling one row of Q touches only that row of the product, so the repair cannot break anything else.

With both in place, all 1944 approximations evaluated in the study were admissible with $\text{max_r} = 1$ exactly.

Choosing the coarse-graining factor L . There is a usable design rule. The *floor* $D_{\text{floor}}(L)$ – the plain max envelope of Eq. (2.4) lifted back to A , with no LP at all – is a hard lower bound for the whole coarse-grained- Q family at any rank, and it is cheap to evaluate. Empirically the achieved distance adds it in quadrature to the distance the same algorithm would reach with unlimited L :

$$D(L) \approx \sqrt{D_{\infty}^2 + D_{\text{floor}}(L)^2} \quad (2.9)$$

which held to $-6\%/+11\%$ over 25 predictions spanning five octaves of 2^{r-L} . So: pick L such that $D_{\text{floor}}(L) \lesssim D_{\text{target}}/3$, which requires only a streamed $\bar{A}$ and one envelope evaluation, and then the LP machinery is worth building.

A second empirical law is useful for extrapolating between configurations. At fixed $n_{\text{dm,wt}} \equiv 2^{r-L}$ (the number of coarse DM bins), D varies by only $1.15\text{--}1.37\times$ over tree ranks 12 to 16, whereas at fixed L it varies by $12\text{--}142\times$. So D is set by $n_{\text{dm,wt}}$, not by L , and in the fine regime

$$D \approx C(r) n_{\text{dm,wt}}^{-0.85}, \quad C \approx 1.3 \text{ to } 2.6 \text{ for } r = 8 \text{ to } 16 \quad (2.10)$$

The exponent is 0.85 and not 1; assuming 1 under-predicts D by roughly $2\times$ per four octaves of extrapolation. Because a fixed `wt_dm_downsampling` is a *finer* fractional DM cut at higher rank, production is better off than any subscale measurement suggests.

Results. All distances below are from `VarianceMap.get_distance()` (§1.2) and all approximations are admissible (`max_r = 1`). “Envelope” is the best plain clustering we found, which is the same greedy agglomerative merge in shape space but with one-hot q_{β} .

One quantity in the table needs defining first. Write `nnz` for the number of *nonzero* entries of q_{β} : how many of the K columns of W a group actually uses. It is a separate budget from the rank – K is the number of *shared* columns of W , `nnz` is how much of it each group draws on – and the two enter the cost differently (see below). The plain envelope is `nnz = 1` by construction, since each group uses one column of W . The Q-step of Eq. (2.6) is *not* asked to be sparse and returns `nnz = 2` to 3 anyway. The “`nnz ≤ 2`” row is the variant where it is additionally constrained: keep the best admissible subset of size 2 of the LP’s own support, then re-run the alternation with that support frozen.

The cleanest measurement is at the real CHIME tree-0 geometry ($r=15$, $N=1$, $P=16$, $N_{\text{freq}} = 16384$, $N_{\alpha} = 524288$, dense $|A| = 64$ GiB), coarse-grained at L corresponding to the shipped `wt_dm_downsampling = 64`, i.e. $N_{\beta} = 8192$ groups. The `GpuDetrenderLps2d` is the one used throughout this study: a single spline zone with $n_{\phi} = 2$, $n = 2$, time half-width 2, $T = 2048$, $\eta = 10^{-3}$, $\epsilon = 3 \times 10^{-5}$, and a uniform knot vector giving 8 knot intervals across the band (hence $N_{\phi} = 10$ spline basis functions), held fixed as N_{freq} varies. **Note the time half-width is 2, which is the pessimistic end of the range:** it is the setting that makes the low-DM rows hard, and the distances below would improve by roughly $2.8\times$ at half-width 8 (see the last bullet of “what is not yet established” below). The table is therefore a conservative operating point, not a tuned one.

rank K	8	16	32	64
(a) envelope (one-hot, <code>nnz = 1</code>)	0.0750	0.0504	0.0357	0.0261
(b) this algorithm (<code>nnz</code> unconstrained)	0.0412	0.0277	0.0191	0.0136
(c) this algorithm, <code>nnz ≤ 2</code>	0.0470	0.0331	0.0235	0.0182
gain at equal rank, (a)/(b)	$1.82\times$	$1.82\times$	$1.87\times$	$1.92\times$
gain at equal rank, (a)/(c)	$1.60\times$	$1.52\times$	$1.52\times$	$1.43\times$

Every entry is a distance D , and all three rows are the same rank K : they differ only in how many columns of W each group may use. Constraining `nnz` to 2 costs 14–34% in D and is worth doing only when it buys more than that in cost, which at this geometry it does not – see below.

For reference, the coarse-graining alone – one weight per group, i.e. rank $N_{\beta} = 8192$ and no factorization at all, which is what production currently computes – gives $D = 0.0043$ on the same map. That is the floor

$D_{\text{floor}}(L)$ of Eq. (2.9).

The trade this exposes is the useful output. Rank 8192 at $D = 0.0043$ costs 1.34×10^8 multiply-adds to apply; rank 32 with $\text{nnz} = 2$ at $D = 0.0235$ costs 5.4×10^5 , i.e. $248\times$ cheaper to apply for $5.4\times$ more D ; rank 8 is $910\times$ cheaper for $11\times$ more D . Which point on that curve is wanted is a science question – how much conservative overestimation of the output variance the peak-finder’s threshold can absorb – not an algorithmic one, and this is the curve on which to answer it.

At equal *apply cost* rather than equal rank, the factorization is $2.4\text{--}3.1\times$ better in D than the envelope at this geometry. On subscale detrended maps the equal-rank gain is $1.23\text{--}1.55\times$, so nothing about the gain was an artefact of small maps – it grows with scale. Restricting the algorithm to see only $\bar{A}$, rather than giving it dense per-row access, costs $0.93\text{--}1.06\times$: nothing, and occasionally better, since the coarse constraint acts as a mild regularizer.

Two further points, which were not obvious to us in advance:

- **It is cheaper to apply, not just more accurate.** Evaluating $y = Q(W^T v)$ costs $KN_{\text{freq}} + N_{\beta} \text{nnz}$ multiply-adds, where nnz is the number of W -columns vectors a group actually uses (the LP returns 2–3, without being asked). At the CHIME tree-0 geometry this buys $2.4\text{--}3.1\times$ in D at equal apply cost. Note the saving comes from coarse-graining Q : with a *per-row* q_{α} the same factorization is *worse* than the envelope at equal cost, because the $N_{\alpha} \text{nnz}$ term dominates. Whether to additionally cap nnz at 2 depends on which term dominates: cap when $N_{\beta} \gtrsim KN_{\text{freq}}/4$, which is true at $N_{\text{freq}} = 400$ and false at $N_{\text{freq}} = 16384$.
- **The output is the array the kernel already stores.** `PeakFindingKernelParams`’ weight array has shape $(n_{\text{dm,wt}}, n_{t,\text{wt}}, P, N)$, i.e. one scalar per coarse-graining group β – which is precisely Eq. (2.2) with $\text{nnz} = 1$. So this algorithm’s Q is the object production already has, generalized from one entry per group to two or three; $\text{nnz} = 2$ means the peak-finder applies the existing weight multiply twice and adds. A *per-row* Q is not expressible in the current kernel at all. The reduction is therefore not only cheaper, it is the only one of the two that fits the hardware.

Two ideas that did not work. Recorded because both look attractive and both cost real effort to rule out.

The “give up on the hard rows” strategy suggested by the saturation of f does not pay. Abandoning a fraction q of rows (sending them to $y \rightarrow \infty$) adds $10q$ to D and removes the share of D those rows carried, and empirically that share is only $\approx 5q$ – the cost is broad, not concentrated. Break-even therefore needs $D \gtrsim 2$, and measured D is below that everywhere; 71 of 72 direct tests lost. The related suggestion of inflating variance at low DM should be dropped for the same reason.

Non-uniform DM cuts (log, sqrt, inverse-log spacings) are no better than uniform ones.

What is not yet established.

- The high-rank ladder ($r > 12$) is all $N = 1$, because no compiled `GpuSbDedispersionKernel` exists for $N > 1$ at `dd_rank` 7–8. The design rules above are statements about the DM axis and are unaffected, but the *absolute* D at a subbanded production config is not measured; the subband ablation suggests it is $1.2\text{--}2.4\times$ higher.
- Nothing at CHORD scale has been measured. The streaming reduction removes the memory obstacle, but scoring D against a fine reference still needs the dense A ; a variant which accepts $(\bar{A}, y, \text{labels})$ would close this, since for a coarse-grained- Q approximation admissibility is a theorem rather than a measurement.
- Everything is single-tree with no early triggers, and no time-downsampled trees.

- The shape-space greedy construction of W is the one step that does not scale: $O(N_\beta^{2.4} N_{\text{freq}})$, extrapolating to days at CHORD’s N_β . Everything else is linear or better. It is a clustering, so a cheaper one would do, but the obvious substitutes (k-means on log-shape) are erratic at group granularity.
- The `GpuDetrenderLps2d` time half-width is a large and cheap lever on all of this: raising it from 2 to 8 improves D at fixed rank by $2.8\times$, removes the low-DM cost concentration entirely, makes the SVD viable again, and costs $\sim 7\%$ more sweep time. Whether that half-width is acceptable on detrending grounds is a separate question, not addressed here.

2.3 Campaign 2 setup [mostly human-authored]

This is really interesting progress! So far we’ve learned that:

1. Coarse-graining $A \rightarrow \bar{A}$ is a good approximation in practice (in the sense that $D(A, \text{lift}(\bar{A}))$ is small). It seems safe to assume that this should always be the first step, and that further steps (such as QW -alternation) should operate on the smaller $\bar{A}$ matrix for speed.
2. QW -alternation is an interesting algorithm for optimizing $D(A^{\text{true}}, A^{\text{approx}})$, given a rank- K initial guess for A^{approx} . As currently formulated, QW -alternation doesn’t change the rank K .
3. Truncating the SVD decomposition doesn’t work on its own. (But I’d like to explore using the SVD decomposition in combination with other ideas – see below.)

In campaign 2, I’d like to explore the following directions:

- I realized that I never specified targets for the distance D and the rank K , so the problem we’re exploring isn’t really well-posed. For now, let’s consider rank $K = 8, 16, 32, 64, 128$. For each of these ranks (and for each choice of dedispersion+detrender config), please explore different algorithms and try to find the smallest D that you can. This will give us a sense of the “Pareto frontier” of distance versus rank.
- While I was discussing the campaign 1 results with an agent, it emerged that the linear programs may have constraints that are not necessary. For example, the Q -step (step 2) may currently have a constraint $q_\beta \geq 0$. I don’t think this is necessary (only the weaker constraint $Wq_\beta \geq \bar{A}_{\beta,:}$). Please investigate further and decide whether constraints can be dropped. Does this significantly improve our results (D -values)? What about the W -step?
- We have an analytic argument (§D) that A has low rank if there is no detrender. If we didn’t know about this analytic argument, one way to “discover” the low rank empirically would be to compute the SVD singular values, and find that most of them are zero (within roundoff).

This made me curious: what does the spectrum of singular values of $\bar{A}$ look like empirically, if there is a detrender? Do we find that the singular values fall off quickly, and a low rank matrix is a good approximation (with respect to the Frobenius norm – not necessarily with respect to our distance function).

- Given a rank- K approximation $A^{\text{approx}} = QW^T$, and $K' < K$, I’d like to explore the following idea for obtaining a rank- K' approximation. First, SVD-decompose $A^{\text{approx}} = Q_0 \Lambda_0 W_0^T$, where Λ_0 is K -by- K diagonal. Then, decrease the rank to K' by dropping the lowest (least significant) SVD modes, obtaining $A^{\text{new}} = Q_1 \Lambda_1 W_1^T$.

As previously discussed, this may ruin the approximation, in the sense that $D(\bar{A}^{\text{true}}, A^{\text{new}}) = \infty$. However, we can use the “ Q -step” above to update $A^{\text{new}} \rightarrow Q'_1 W_1$, where Q'_1 is a new matrix obtained by solving a linear program, and W_1 is unchanged. Then, $D(\bar{A}^{\text{true}}, A^{\text{new}})$ will be finite (under mild assumptions on W_1) and we can continue the QW -alternation algorithm from here.

Similarly, we could also keep Q_1 fixed and use the “ W -step” to update W_1 – I’m not sure which variant works better.

Note that before trying this idea, you may need to remove unnecessary positivity constraints from the LP (see first bullet point above), since SVD modes have arbitrary signs.

- Combining ideas from the previous two bullet points, we could also try initializing the low-rank decomposition $A^{\text{approx}} = QW^T$ using the SVD decomposition of $\bar{A}$ (followed by QW -alternation), instead of the clustering algorithm above (“Step 1: constructing W ”).

This could either be done by keeping K singular values, where K is the desired rank, or following a “schedule” where a larger number of singular values are kept, and we alternate between QW -alternation (to reduce the distance) and reducing the rank (using the idea from the previous bullet point).

- I’m wondering whether the SVD decomposition is also useful for setting up the LP problems in the Q -step and W -step. For example, suppose that we have a decomposition $A^{\text{approx}} = QW^T$, and we want to do a Q -step.

Suppose we SVD decompose $A^{\text{approx}} = Q_0\Lambda_0W_0^T$, where W_0 is semiorthogonal ($W_0^TW_0 = 1$), and then solve an LP to obtain $A^{\text{new}} = Q_1W_0^T$. Is this more numerically stable, because we have orthogonalized columns of W_0 to avoid near-cancellations? Are there other technical advantages, such as a more principled way to set numerical thresholds in the LP? Please investigate.

Another idea: if we find empirically that some singular values in Λ_0 are very small, then it may make more sense to drop these modes (decreasing the rank), since the extra modes aren’t “helping” the approximation.

- Another idea for initialization (in addition to the clustering algorithm or the SVD decomposition). Let $\bar{A}_0$ be the coarse-grained variance map with the detrender removed (but the dedispersion config unchanged). Using the algorithm in §D, we have an exact low-rank decomposition $\bar{A}_0 = Q_0W_0^T$. Suppose that we fix W_0 and use a Q -step to approximate $\bar{A} \approx Q_1W_0^T$. Is this a useful starting point for obtaining a low-rank approximation?

More generally, if you have any other “natural” ideas for generating a W -matrix (even if there is no obvious way to generate Q beyond solving an LP), this could be interesting to investigate.

- A small idea that may help speed up the LP solver. Currently, we solve with a subset of the constraints for speed, then re-evaluate all constraints. If any constraints fail, then we add constraints and re-solve. Suppose that when we evaluate the constraints, we find that they only fail by a small amount (say $A^{\text{approx}} \geq (1-\epsilon)A^{\text{true}}$ everywhere). Then we could solve the constraints by simply rescaling the solution by $(1-\epsilon)^{-1}$ – the only thing we lose is that the distance is slightly suboptimal. In early stages of an iterative algorithm (either QW -iteration or a larger iterative algorithm involving SVD decomposition and rank reduction) this may be a win – it may be better to trade strict optimality for speed in the early stages.
- So far we haven’t analyzed dedispersion configs with **subband_counts**. Do our results (1 and 2 above) hold up in this case? Does the distance-rank “frontier” get worse in the presence of **subband_counts**?
- Similarly, how do our results depend on W ? So far, we’ve only analyzed $W = 2$. Note that $W = 0$ implies $n = 0$.
- Please be creative and pursue your own ideas, or extend/generalize the ideas here. As you explore the problem, new ideas may suggest themselves. Sometimes an experiment is useful if it improves general understanding of the problem domain, even if it doesn’t immediately lead to a concrete improvement in the algorithm.

In addition, here are some directions to explore that were suggested by an agent:

- **The GpuDetrenderLps2d knot vector as a design variable.** Campaign 1 found that the detrender does not merely perturb the structure of A , it destroys it: with no detrender, 33% of a subbanded A is exactly zero (a multiplet gets no contribution from channels outside its subband), and adding a single-zone detrender takes that to 0.0%. But a detrender whose spline zones are *aligned with the frequency zones* keeps 25% of the zeros. So the sparsity that a low-rank construction would exploit is partly a *choice* (the knot vector), not a given.

This suggests a direction that changes the problem rather than the algorithm: are there knot vectors which leave A substantially more approximable at fixed rank, and what do they cost in detrending quality? The time half-width W is one detrender knob and is already covered above; the knot vector is a second, independent one, and nothing has explored it. If the cost is small this may be the cheapest available win.

- **Rank reduction guided by D rather than by singular value.** This is the natural competitor to the SVD-truncation idea above, and the two should be measured head to head in the same experiment. Instead of dropping the modes with the smallest singular values, drop the column of W whose removal – followed by a Q -step – increases D least, repeating until the rank is K' (a “leave-one-out” criterion).

The motivation is that a singular value measures a mode’s contribution in Frobenius norm, which is the wrong norm here: campaign 1 found that pruning W by column *usage* was $2.1\times$ worse than simply rebuilding at the target rank, and diagnosed it as keeping “columns that supply mass, not columns that supply shapes nothing else covers”. A singular value is also a mass-like criterion. A leave-one-out score is aligned with the objective by construction. It costs K Q -steps per removal, but those are independent and each is cheap.

- **Pivoted QR (or CUR) for choosing the columns of W .** Campaign 1 tried using actual rows of A as columns of W , chosen worst-covered-first, and found them $1.2\text{--}1.4\times$ worse than the greedy clustering. That may say more about the selection rule than about the idea: a rank-revealing QR with column pivoting applied to $\bar{A}^T$ chooses a well-conditioned subset of rows, which is the principled version of the same construction.

It has several practical advantages worth testing: it is deterministic, it costs $O(KN_\beta N_{\text{freq}})$ rather than the $O(K_0^{2.4})$ of the agglomerative greedy, the atoms stay nonnegative and interpretable (each is a real output’s frequency response), and the pivot order gives a natural nested sequence of W -matrices – which is exactly what a rank schedule wants.

- **Warm-starting the linear programs.** The Q -step solves N_β independent LPs, and the whole thing is repeated across alternations and again at every rank in a schedule. Consecutive solves differ only slightly, so a dual-simplex warm start from the previous optimal basis should be a large constant factor. Neighbouring groups are also similar to each other (adjacent coarse DMs have similar rows), so one group’s basis can seed the next.

This is complementary to the constraint-generation idea above rather than an alternative to it, and it may well be the larger of the two effects. Worth measuring as LP wall clock per Q -step, with and without.

- **Does a W -matrix transfer between trees and between configs?** Building W is the one step of the recommended algorithm that does not scale: $O(K_0^{2.4} N_{\text{freq}})$, which extrapolates to days at CHORD’s K_0 , while everything else is linear or better. But a `DedispersionPlan` has several trees, and production needs weights for all of them.

So: does a W -matrix built for one tree work for another (same N_{freq} , different rank or subband structure)? If it does, the expensive step amortizes over trees, and possibly over configs. There is some reason for optimism – campaign 1 found W need only be *reasonable* rather than optimal (a cheap k-means W matched the greedy at $K = 32$). Note the related question of transfer across *granularity* within one tree is already settled: building W from $\bar{A}$ at the same L used for Q is within a few percent of building it from a finer $\bar{A}$, so there is no reason to keep a finer one around.

2.4 Campaign 2 results [agent-authored]

Campaign 2 ran fourteen experiments against the questions above. The short version is that the algorithm of §2.2 was replaced in two places, is now 2–3× better at every production rank, and has stopped being limited by rank at all. Two of the bullet-point directions turned out to be dead ends and are reported as such.

The recipe, as it now stands. Three of the four steps of §2.2 survive unchanged: coarse-grain $A \rightarrow \bar{A}$ by a max over structured groups, solve one covering LP per group for q_β , and alternate with the W -step. What changed is how W is built, and the *constraint set*:

1. **W is now a truncated SVD of $\bar{A}$** , not an agglomerative clustering. Use the SVD of the unit-sum *shape* matrix $S_{\beta F} = \bar{A}_{\beta F} / \sum_F \bar{A}_{\beta F}$ for $K \gtrsim 32$ and of $\bar{A}$ itself below that. Only at $K \leq 8$ does the clustering still win.
2. **The Q -step constraint $q_\beta \geq 0$ is dropped.** It was never implied by the problem – admissibility constrains the *product* $Wq_\beta \geq \bar{A}_{\beta,:}$, not the coefficients – and dropping it is worth 1.4–3.9× in D at fixed rank, like for like. The W -step’s $W \geq 0$ may be kept when the initial W is nonnegative (it is inactive at the optimum, worth 0–2.6%), but must *not* be imposed on top of a signed initializer, where it costs 8.8×.

The bullet points above asked whether the SVD could be used for initialization, for rank reduction, and for conditioning. The answers are yes, yes, and no – but the reason the first answer is yes turns on a distinction the campaign took a long time to draw, and which is worth stating plainly because it is the single most useful idea to come out of this work:

The SVD as an *approximation* and the SVD as a *W-matrix* are different objects.

A truncated SVD used directly as A^{approx} has an admissibility cliff: below $K \approx 32$ it puts a non-positive value on a positive entry of $\bar{A}$, which no rescaling can repair, so $D = \infty$. The same truncation used as the *W-matrix*, with the LP free to choose q , has no cliff at all.

These two are easy to conflate, and conflating them is why the winning algorithm was nearly abandoned: measurements of the first were repeatedly read as verdicts on the second. Note also that the two changes are coupled. A signed W is unusable while $q_\beta \geq 0$ is imposed – the sign convention of an SVD mode is arbitrary, and with q constrained the LP was infeasible on 400 of 400 sampled rows. With q free the problem is exactly invariant under $W_{:,c} \rightarrow -W_{:,c}$, $q_c \rightarrow -q_c$, so the sign convention stops mattering (measured: identical D_0 to 17 digits). Dropping the constraint is what made an SVD W -matrix available at all.

The frontier. At the real CHIME tree-0 geometry ($r = 15$, $N_{\text{freq}} = 16384$, coarse-graining $L_0 = 6$), with the recipe above:

rank K	8	16	32	64	128
D , campaign 1	0.0412	0.0277	0.0191	0.0136	0.0105
D , campaign 2	0.0376	0.0147	0.00686	0.00533	0.00462
improvement	1.10	1.88	2.78	2.56	2.28
D/D_{floor}	8.52	3.34	1.55	1.21	1.05

where $D_{\text{floor}} = 0.00441$ is the coarse-graining floor: the covering constraint forces $y_\alpha^{\text{approx}} \geq \sum_F \bar{A}_{\beta F}$ for every $\alpha \in \beta$, so *no* approximation with a coarse-grained Q , of *any* rank, can beat the unrescaled max-envelope over the L_0 groups. The last row is the important one.

Rank is spent; the coarse-graining is the remaining lever. At $K = 128$ we are 4.8% above the floor, so further rank buys almost nothing. This was initially misread. The frontier’s log-log slope softens steadily with K (from -0.57 to -0.22), which looked like an unexplained bend, and the naive diagnostic – “ D is still $2.4\times$ the floor, so the floor is not binding” – is simply wrong reasoning. **The floor enters additively**, and therefore governs the slope long before D approaches it. Subtracting it leaves a clean power law,

$$D(K) \simeq D_{\text{floor}} + cK^{-2} \quad (2.11)$$

which fits over a factor 16 in rank and predicted $D(K=128)$ to 0.5% before it was measured. The right measure of “how floor-limited is this cell” is $1 - D_{\text{floor}}/D$, not D/D_{floor} .

Since D_{floor} is set by L_0 , the lever is L_0 . Refining it from 6 to 4 buys $2.10\times$ at $K = 64$, against at most $1.21\times$ available from *any* rank at $L_0 = 6$ – about $7\times$ more per unit of storage. The crossover is sharp and production has just crossed it: at $2\times$ storage from $(L_0, K) = (6, 32)$, buying rank wins $1.45\times$ to $1.26\times$; from $(6, 64)$, buying groups wins $2.10\times$ to $1.15\times$. Below $K \approx 29$ buying groups is a $3\text{--}7\times$ mistake. **So: refine L_0 first, then buy rank** – and the campaign was right to optimize rank until the new W -matrix landed.

The axis is short, and the limit is in the shipped code rather than in the mathematics: `DedispersionPlan` accepts `wt_dm_downsampling = 64, 32, 16` but fails an assertion at 8, because `dm_downsampling` defaults to 2^R . Hence $L_0 \geq R$, giving exactly two refinements; after them D/D_{floor} is back to 2.3 and rank binds again at $K \approx 75$.

Cost: the one step that did not scale is gone. The clustering construction of W was $O(N_\beta^{2.4} N_{\text{freq}})$ and extrapolated to days at CHORD. The SVD W is one Gram eigendecomposition: 34 s against ~ 2.5 h at subscale, and 120–730 s against ~ 44 h at the production point. Everything else was already linear or better. Separately, W turns out not to need *fitting* at all: one W -step takes a W -matrix built for a *different* tree, subband count or detrender to within 3–9% of a purpose-built one, and the analytic no-detrender factorization of §D costs only 1.0–1.8 $\times$ with no clustering whatever. A *random* W costs 1.2–2.0 $\times$, so the choice still matters – it just needs no expensive search.

Dependence on the configuration.

- **Subbands are expensive:** $D(N)/D(N=1)$ is 3.1–3.8 $\times$ at $N = 9$ and 5.7–22.8 $\times$ at $N = 16\text{--}20$, far worse than campaign 1’s 1.2–2.4 $\times$ ablation estimate. At the CHORD-*shaped* subscale point ($r = 15$, $N = 25$, 400 channels — CHORD’s subband shape at a rank and channel count we could afford, *not* the CHORD config, which is $r = 16$ at 28160 channels) the recipe reaches $D = 0.047, 0.028, 0.014$ at $K = 32, 64, 128$.
- **The detrender half-width W is a real lever and does not saturate at 8:** $W = 8$ buys $2.5\times$ at $K = 64$ over $W = 2$, and $W = 16$ a further $1.7\times$. Whether such a window is acceptable on detrending grounds is still a separate question.
- **The knot vector is a weak lever, but not a dead one.** Aligning spline zones with *frequency* zones is worth nothing. Aligning them with *subband* boundaries is worth $1.1\times$ at $K = 64$ and $2.5\times$ at $K = 128$. What that alignment costs in detrending quality is not answerable from A alone.

Two methodological warnings, both learned the hard way. These cost the campaign more than any single measurement, and both should be applied to anything in this section:

1. **A conclusion measured at $N_{\text{freq}} = 400$ may reverse at 16384.** It happened three times: the SVD-versus-LP comparison, the subband rank-budgeting rule, and “the W -step is redundant”. The mechanism is understood – the LP family gets $\sim 3\times$ *better* with more channels while the SVD family gains nothing – but the practical rule is simply to treat any subscale recommendation as provisional.

2. **A floor-based conclusion is not an LP-based conclusion.** The coarse-graining floor ranks the W ladder perfectly and the knot-vector axis *oppositely*: one configuration with a $19\times$ better floor is $1.3\times$ worse in achieved D . Use the floor as a bound, never as a ranking. The singular-value spectrum of $\bar{A}$, by contrast, *does* rank configurations correctly (Spearman $\rho \approx 0.97\text{--}0.99$) for the cost of one eigendecomposition, and is a good cheap triage – though it too degrades above $K \approx 64$, where the floor takes over.

What is still open. The largest item is not a physics question but a solver one. The covering LPs are solved by HiGHS, and at high rank on heavily subbanded maps a small fraction of them ($\lesssim 1\%$) return a point that violates their own constraints while reporting success, or fall back to a crude seed. This is not a rounding-level nuisance: one failure per ~ 450 groups costs more D than an entire doubling of the rank, and it is why the refined- L_0 , $K = 128$ cells are currently estimates rather than measurements. A repair is known (halve the rank, re-solve, keep the rank) and takes seconds; until it is in place, any number at $K \geq 128$ on a subbanded map should be treated as provisional. Beyond that: warm-starting the LPs was never attempted, and nothing has been measured at CHORD’s channel count.

2.5 Campaign 3 setup [human + agent authored]

Directions for the next campaign. (The rest of the campaign-2 leftovers – the solver defect at $K \geq 128$, the exact $(L_0, K) = (4, 128)$ cell, LP warm starts, and CHORD’s channel count – are tracked outside these notes.)

The campaign 2 results look promising! The main things I’m worried about are **scalability** and **robustness**. Consider the CHORD scale config `chord_sb2.yml` (primary tree 0 only), with $N_{\text{freq}} = 28160$, rank $r = 16$, and 25 subbands.¹ Can we reach this scale in a reasonable amount of computing time (say < 1 day on 32 cores)? Will our current algorithm run into numerical instability (e.g. in the LP solver) before reaching this scale? What rank- D “Pareto frontier” will we see at CHORD scale?

- Instead of running `chord_sb2.yml` directly, approach it via a sequence of increasingly large-scale configs (e.g. increasing r by 1 and/or doubling N_{freq} , and/or adding subbands. You may find that new issues emerge as the scale increases.
- Try to reach as large a scale as possible, within the time horizon of the ch-research skill. In order to achieve this goal, consider carefully what set of parallel subagents will make best use of our 32 cores and 2 GPUs. For example, if you’re precomputing A -matrices on the GPU and solving LPs on the CPU, you may want to precompute the next problem instance in parallel with solving the current instance.
- You may not reach CHORD scale within the time horizon. In that case, please try to find an empirical scaling law based on your subscale results, and predict what “Pareto frontier” we might see at CHORD scale.
- This idea is cut-and-paste from an earlier section – I’m including it again since it may be more helpful at scale.

A small idea that may help speed up the LP solver. Currently, we solve with a subset of the constraints for speed, then re-evaluate all constraints. If any constraints fail, then we add constraints and re-solve.

Suppose that when we evaluate the constraints, we find that they only fail by a small amount (say $A^{\text{approx}} \geq (1-\epsilon)A^{\text{true}}$ everywhere). Then we could solve the constraints by simply rescaling the solution by $(1-\epsilon)^{-1}$ – the only thing we lose is that the distance is slightly suboptimal. In early stages of an

¹This said $r = 15$ when the campaign was set up, and that was wrong: primary tree 0 has `ipri == 0`, so it keeps `toplevel.tree.rank`, which is 16. Trees 1–3 are $r = 15, 14, 13$. Measured off `DedispersionPlan` by two agents independently, and confirmed byte-exactly by the size of the resulting $\bar{A}$. It matters: the true geometry is $n_{\text{dm}} = 4096$, $M = 91$, $P = 16$, $N_\alpha = 5\,963\,776$, so $N_\beta(L_0=4) = 1\,638\,400$ – twice what the campaign first assumed, and every LP cost scales with it.

iterative algorithm (either QW -iteration or a larger iterative algorithm involving SVD decomposition and rank reduction) this may be a win – it may be better to trade strict optimality for speed in the early stages.

- **Pin a nonnegative column in W , and exclude it from the W -step.** Here’s a minor suggestion that may be useful.

Campaign 2 dropped $q_\beta \geq 0$ entirely and, under the SVD recipe, dropped $W \geq 0$ as well, so both factors are now sign-free. That is where the performance came from, but it quietly removed something several parts of the algorithm still rely on: *the existence of at least one all-positive column of W* . Three places currently hope for one rather than requiring it – the additive admissibility repair (which raises the product in every channel at once only if the column it adds is nonnegative), the one-hot seed for the first Q -step, and the argument that a covering LP is feasible at all. Today that hope is usually satisfied only because the leading SVD mode of a nonnegative matrix is itself nonnegative (Perron–Frobenius), which is a fact about *one* mode and is not guaranteed to survive truncation, shape normalization, or a differently-constructed W .

The proposal is to stop relying on it: reserve one column of W , fix it to a nonnegative vector, and exclude it from the W -step so it cannot drift. The cost is bounded by one unit of rank, and is probably less – see below.

Which vector. An all-ones column is the obvious choice and is the wrong one. It is nonnegative and dominating, but its cost in the objective is $s_c = N_{\text{freq}}$, so covering a group whose largest entry is m costs mN_{freq} against a true row sum of $\sum_F \bar{A}_{\beta F}$ – i.e. one pays the row’s peak-to-mean ratio. On a subbanded map a row is supported on roughly $1/N$ of the band, so at CHORD’s $N = 25$ the all-ones column is $\sim 25\times$ more expensive than the row it is covering, and the LP would never choose it except in extremis. A better choice is the elementwise column max over groups,

$$W_{F,c_0} = \max_{\beta} \bar{A}_{\beta F} \quad (2.12)$$

which is nonnegative by construction, is positive in exactly the channels where some group needs it, costs one streaming pass to build (the same pass that already produces $\bar{A}$), and is the *tightest* single vector that dominates every group. It is also the rank-1 member of campaign 1’s envelope family, so unlike an all-ones column it is a vector the LP will actually use on hard groups.

What it guarantees. With Eq. (2.12) pinned, $q_\beta = e_{c_0}$ is a certified feasible point for *every* group, with a nonnegative product, whatever the other $K - 1$ columns look like. We currently have no such certificate, which is why campaign 2 had to prove feasibility by hand to establish that the solver was reporting a feasible LP as infeasible. It also means the one-hot seed can never fail to exist, and the additive repair can never run out of usable columns.

What it does not fix. It does not prevent the product Wq_β from going negative in channels where $\bar{A}_{\beta F} = 0$ – that is what constraining every channel is for – it only makes such a case repairable. And it does not address the solver’s rank-dependent failures at $K \geq 128$, which are a conditioning problem, not a feasibility one. It is complementary to both: a certified feasible point is exactly what one would hand the solver as a warm start.

The measurement. Compare the current recipe at rank K against the pinned variant at the same rank K (so $K - 1$ learned columns plus one pinned), on at least one subbanded and one unsubbanded map, reporting D and the LP failure counts. The interesting question is whether the pinned column costs a full unit of rank or a fraction of one: the leading SVD mode is already approximately the mean spectrum, so a pinned envelope may be largely reconstructing something the truncation would have spent a mode on anyway.

- Does it help (e.g. with numerical instability at scale) to “re-orthogonalize” the rank- K approximation after each Q -step or W -step, by taking the SVD decomposition? (Note: if we decide to “pin” one or

more all-positive columns, as suggested in the previous bullet point, then we’d want to exclude pinned columns from the re-orthogonalization.

- Feel free to pursue any loose ends from campaigns 1 and 2 that weren’t fully resolved, or that you want to revisit. In particular, ideas that were abandoned previously in subscale instances may prove to be useful at larger scale.
- As usual, please get creative and explore your own ideas!

2.6 Campaign 3 results [agent-authored]

Campaign 3 ran thirteen experiments and answered its question in the affirmative: the recipe reaches CHORD scale, the frontier there is *measured* rather than extrapolated, and the binding constraint turned out to be none of the things the setup worried about.

The answer. The real config was swept end to end: `chord_sb2.yml` primary tree 0, 28160 passes, 5.6 hours on one L40S, with the dense A never formed. At the resulting map, with $L_0 = 6$ ($N_\beta = 409\,600$), a frozen W and 414\,208 covering LPs – all of which returned **optimal**:

rank K	8	16	32	64	128
D	0.686	0.134	0.107	0.0651	0.0236
D/D_{floor}	453	88	71	43	15.6
one Q -step, core-hours	43	69	280	1049	2823
<i>with constraint generation</i>	1.8	2.2	12.7	64	504

with coarse-graining floor $D_{\text{floor}} = 0.00151$. High ranks are an unbiased Monte-Carlo over a random subset of groups, with standard error 0.5–2%; the rest are exact. **The question as posed – what fits in under a day on 32 cores – is answered $K \approx 150$.**

The $K = 128$ value landed inside a prediction of 0.023–0.027 that had been registered in advance, from a different tree geometry at one eighth of the channel count. That is the second pre-registered prediction of the campaign to hold; the first was the sweep time itself, predicted at 6.5 ± 0.5 hours by extrapolating a ladder and then measured at 5.6.

Nothing that was expected to be hard was hard. The dense A at this config would be 1.2 TiB, so the dense-free construction of campaign 1 is what makes any of this possible – but with it, **the sweep is noise**: 5.6 GPU-hours against an LP stage two to three orders of magnitude larger. The GPU kernel the config needs was already compiled, and `dd_rank` turns out to depend only on `toplevel_tree_rank`, not on the channel count, so no new kernel was ever needed. Memory was a real threat and was removed by one fix: the admissibility repair was forming a dense ($N_\beta \times N_{\text{freq}}$) product with a same-sized temporary up to ten times, which is ~ 900 GiB of transient at $L_0 = 4$; blocking it over rows is bit-identical, $1.4\times$ faster, and flat at 0.42 GiB.

CHORD and CHIME say opposite things, and CHIME was the misleading one. Campaign 2 closed on *refine L_0 , then buy rank*, because at the CHIME point $K = 128$ sits $1.05\times$ above the coarse-graining floor – rank was spent. At CHORD, $K = 128$ is $15.6\times$ above the floor, refining L_0 is worth $\pm 2\%$ at *every* rank, and the $64 \rightarrow 128$ doubling still buys $2.8\times$. **At CHORD, rank binds and L_0 is dead.** This is the exact inverse of the previous recommendation, and it is why the campaign’s remaining effort went into making rank affordable rather than into better approximations.

Two conclusions inherited from campaign 2 did not survive contact with the real scale. Both had been measured at one subband and were being applied at twenty-five.

1. **“Subscale numbers are conservative” is false at CHORD’s subband count.** The LP was measured to improve $\sim 3\times$ from 400 to 16384 channels at $N = 1$, and that licence was used throughout two campaigns. On a paired six-rung ladder at $N = 25$, D instead gets *worse* by up to $5\times$ before saturating. So every 400-channel subbanded number in these notes is anti-conservative, in the opposite direction to what was assumed. The mechanism was measured: the shape spectrum flattens by $11\times$ with channel count.
2. **Sweep cost does not predict D cost, on any axis.** Going from the subscale proxy to the real geometry involves three changes, and their measured effects are unrelated to their measured sweep costs: the peak-finding profile count P ($1 \rightarrow 16$) is nearly free to sweep and is the largest accuracy *gain*; the tree rank ($15 \rightarrow 16$) costs $4.1\times$ to sweep and is worth $1.01\times$ in D at $K = 128$; the five frequency zones are free to sweep and *reverse sign*, helping at low rank and costing $1.5\times$ at $K \geq 64$.

This is the same trap as campaign 2’s “a floor-based conclusion is not an LP-based conclusion”, in a new disguise: a cheap proxy that orders configurations correctly on one axis need not order them at all on another.

The largest single win had been sitting unclaimed for two campaigns. The covering LP has one constraint per frequency channel – 28160 of them for K variables – and almost all are slack at the optimum. Constraint generation was proposed in campaign 1, measured at $8.7\times$, and never adopted. Brought forward unchanged it is a *loss* of $0.67\text{--}0.99\times$ at $K \geq 32$; the missing ingredient is a **pool of binding constraints shared across groups**, and with it the Q -step is $5.6\text{--}32\times$ **cheaper with D unchanged to 8–13 digits**. It is a $2.3\times$ loss below ~ 1500 channels, which is why the original subscale measurement undersold it.

Boundedness is what makes this legal and is easy to get wrong. Dropping constraints from a free-sign LP creates a recession direction: sampling constraints and never adding them back leaves the LP *unbounded* on 896 of 896 groups. Keeping one aggregate row of the omitted constraints fixes it completely ($26/32$ relaxations unbounded without it, $0/32$ with).

Two numerical defects, and neither is where we looked first. Campaign 2 left a picture in which the solver was at fault. The solver is not at fault.

- **The multiplicative repair.** When the LP returns a slightly inadmissible point, the repair rescales a whole row (or column) upward – and D is precisely a penalty on scaling things upward. Replacing that rescale by a rank-preserving *additive* lift restores identical admissibility for a relative change of 10^{-9} instead of percents. This is a single function acting on two axes: on the Q -side the damage grows with *rank* (up to $1.6\times$ at $K = 384$, and it removes a provably-impossible non-monotonicity in which $K = 384$ scored worse than its own $K = 256$); on the W -side it grows with *group count* (up to $2.5\times$), because that multiplier maximizes over N_β rather than over N_{freq} .
- **The dictionary column scale.** Rescaling the columns of W is provably inert – it cannot change the feasible set or the objective – yet it is worth up to $1.5\times$ in D . **The mechanism is not understood.** The natural explanation, that the columns sit far from the solver’s absolute feasibility tolerance, was tested and falsified: a $64\times$ change in that tolerance moves D in the eighth digit. This is the campaign’s most interesting loose end.

Separating these two took most of the campaign, and one wrong hypothesis along the way (that they were the same defect) was refuted by measuring them crossed at fixed channel count.

Freeze the W -step – for cost, not for stability. Whether to alternate at all turned out to depend on two axes that point in opposite directions: more channels help the W -step, more groups hurt it. At the real config, with the additive repair, it would help slightly (1.1–1.2 $\times$). It is still not worth running: $K = 64 \rightarrow 128$ buys 2.8 $\times$ for 1774 core-hours where a W -step buys $\sim 1.1\times$ for ~ 1460 . **Rank is simply the better purchase**, which is a more comfortable reason to freeze than the instability that motivated the question.

The two ideas proposed in §2.5. *Pinning a nonnegative column* works and is cheap insurance rather than a performance win: it costs a median 0.23 columns of rank, and it guarantees by construction the nonnegative column that the seeding and repair machinery needs. That guarantee matters more than expected – with `numpy`’s raw SVD signs there are *zero* nonnegative columns and the one-hot seed fails on every map tested; canonicalizing signs recovers exactly one. One correction to the reasoning in the setup: the pinned envelope duplicates the leading SVD mode only when $N = 1$, so it should be added *to* that mode, not substituted for it.

Re-orthogonalization is a no: it is provably a no-op on D , and empirically unreliable, helping at $K = 64$ and hurting by 3 $\times$ at $K = 128$. The factored formulation proposed in the setup – orthogonalize the two tall-skinny factors separately and carry a $K \times K$ middle matrix – is correct and removes the memory objection entirely (1.7 GB against ~ 690 GiB), but it lands *on* the column-scaling above rather than past it: putting Λ entirely in Q reproduces plain column normalization to eleven digits, and no other placement beats it. The middle matrix should be folded away, for a reason worth recording: the rotation destroys the nonnegative column, which no choice of Λ recovers.

What is still open. The frontier above $K = 128$ has never been measured at the real point, and is now affordable. The mechanism behind the column scaling is unknown. The W -step recommendation rests on an extrapolation from 16384 groups to 409600 that one further cell would settle. And the channel-count trend turns over somewhere between 7040 and 28160 channels, which nobody has resolved.

2.7 Campaign 4 setup [mostly human authored]

Between campaigns 3 and 4, we did some refactoring and cleanup. We started developing a new subpackage `pirate_frb.varmap`, that will eventually supersede some of the code in `slow_avar/` and `research/.../*.py`.

The central class in `pirate_frb.varmap` is a flexible `VarianceMap` class. A lot of our existing code is naturally refactored around this class. For example, the “brute force” algorithm for computing variance maps can be refactored as a function that returns one or more `VarianceMaps` (rather than new classes `BruteForceVarianceMap`, `GpuBruteForceVarianceMap`). Building blocks of the low-rank approximation algorithm (e.g. SVD initialization, Q -step, W -step) can be refactored as functions whose inputs/outputs are `VarianceMaps`, or as `VarianceMap` methods.

Eventually, we want the `varmap` sub-package to supersede the following code (hereafter “old code”):

- In `slow_avar`: `brute_force/*.py`, `variance_map_io.py`, `VarMapDistance.py`, `varmap_eval.py`
- All code in `research/toplevel/*.py`.

The following code is NOT in the scope of this refactor (hereafter “new code”):

- In `slow_avar`: `check/*.py`, `PfVariance.py`, `SparseTile.py`, `TmpVmap.py`

Assume that files in the previous bullet point will not be changed, and feel free to call code in these files from new code in `varmap`.

The main goal of `varmap` is NOT to implement our current “frontier” algorithm as expediently as possible. The main goals are **flexibility** and **generality**, since we’re still in the process of exploring algorithms. A

good approach is to implement low-level “building blocks” that can be called sequentially from a script to create a high-level algorithm. Simplicity and readability are also important – this is an opportunity to clean up historical baggage. So please think carefully “from scratch” about whether existing code could be simplified, cleaned up, or improved.

Currently, the refactor is logically complete, and we’re nearly ready to delete the old code. Part of the purpose of campaign 4 is to try the new code (`pirate_frb.varmap`) at scale. For now, I’m keeping around the old code as a reference. If campaign 4 goes smoothly, then the old code will be deleted.

In campaign 4, please do the following:

- The ultimate goal of campaign 4 is to create an on-disk `VarianceMap` at rank $K = 128$, for `chord_sb2_et.yml`, base tree (tree 0) only, and to collect timings and D -values (distance function) for each intermediate step. This will search as a benchmark for future campaigns, to try to improve either the timing or the final D -value.
- A secondary goal is to curate a well-defined “ladder” of subscale examples, which are similar to `chord_sb2_et.yml` (single-tree), but increase in scale by roughly a factor ~ 2 . Use your judgement to decide how to define the ladder.

For each example in the ladder, also collect timings and distance functions (as in the previous bullet point). This will also be part of our benchmarking suite for future campaigns.

- Throughout campaign 4, do not use any old code (see above) unless absolutely necessary. Instead, use the equivalent code in `pirate_frb.varmap`. I haven’t deleted the old code yet, in case it is useful for debugging.
- Normally, the toplevel agent is forbidden (via a rule in the `ch-research` skill) from modifying source files. During campaign 4, this rule is modified as follows. The toplevel agent may modify source files, but only in the `varmap` subpackage, only if no subagents are running (in order to ensure that files are not modified while in use), and only if the modification is an unambiguous improvement (such as a bugfix).

Bugfixes and improvements may be suggested by subagents (in addition to the toplevel agent). The toplevel agent should make a judgement call whether to “drain” subagents in order to edit code (since code cannot be edited while subagents are running), or continue to spawn subagents and deprioritize code edits.

The purpose of this rule is to allow debugging during campaign 4, since the new code may not fully be debugged. Note that in order to fix bugs, you’ll need to “drain” running subagents – use your judgement to decide whether this is sensible.

- During campaign 4, don’t explore new ideas or implement new features, but do feel free to experiment with different settings, and different features that are already implemented. The purpose of campaign 4 is to establish a baseline, using ideas that we’ve already implemented, for future improvements – not to explore new ideas.
- At the end of campaign 4, make a recommendation: should we delete the old code, or is there still value in keeping it?

Update (after campaign 4). The old code has been deleted. The `pirate_frb.slow_avar` subpackage is gone, and with it `PfVariance.py`’s `PfVariance` / `PfAvarExact` / `PfAvarApproximation` classes, `TmpVmap.py`, and the `check_*.py` CLI entry points – so the “new code” bullet above no longer describes anything on disk. Two pieces of it were kept, because they are substrate rather than an alternative route to the same answer, and both moved into `varmap/`: `SparseTile.py` (minus `SparseTilePerM`, which had no consumer outside a test oracle) and `PfVarianceConvolver`. On the C++ side, `struct PfVariance` and `class PfAvarApproximation` were removed from `src_lib/varmap.cpp`, and the `fast_avar` python subpackage was renamed `fast_varmap`.

2.8 Campaign 4 results [agent-authored]

Campaign 4 ran nine experiments over 24 hours and did what it was asked: it built the rank-128 CHORD map with `pirate_frb.varmap`, timed every step, curated a ladder, and came back with a recommendation on the old code. Nothing failed. The headline is that **the port reproduces at production scale**; the more useful results are three things the campaign learned on the way, each of which changes how a future campaign should read its own numbers.

Two facts made the campaign cheaper than planned. First, **tree 0 of chord_sb2.et.yml is byte-identical to tree 0 of chord_sb2.yml** – the configs differ only in `num_early_triggers` for trees 1–3 – so campaign 3’s archived sweep was reusable and no brute-force sweep was needed, saving ≈ 5 GPU-hours. Second, $L = 6$ is not merely a good grouping but exactly the production one: $N_\beta = \text{ndm_wt} \times N \times P = 1024 \times 25 \times 16 = 409\,600$.

The deliverable. An on-disk rank-128 VarianceMap for `chord_sb2.et.yml` tree 0, at $L_0 = 6$:

D	0.024308952727723543 , nscored = 5 963 776
file	research/agent037/chord.t0.L6.K128.asdf, 473.1 MiB
Q -step cost	479.7 CPU-core-hours measured, 9.02 h on 60 workers
end-to-end from the seed	513.6 CPU-core-hours
solver	409 600 of 409 600 optimal ; zero failures, zero rescues
admissibility	max_r = 0.9999999999886, nviol = 0 of 1.15×10^{10}
read-back	5.6×10^{-14} relative
D before the Q -step	6.378 (the SVD seed), so the step buys a factor 262
D/D_{floor}	16.1

The intermediate steps: `coarse_grain(6)` of the archived $L = 4$ map took 801 s and produced an 85.98 GiB reference; `svd(128)` took 392 s (2.22 core-hours), with $W^T W = I$ to 2.4×10^{-15} ; the serial repair, which does not parallelise, is ≈ 69 min at this geometry.

Against campaign 3’s published $D = 0.023594$ for the same cell this is +3.03%, and against its 504 core-hours, +1.9%.²

D is reproducible to 6%, not to machine precision. This is the campaign’s most important methodological result, and it was measured rather than assumed. **Two solver arms whose LP objectives agree to 9×10^{-14} give distances differing by 6.0%.** A degenerate covering LP has many optima with the same $s \cdot q$ and very different violation profiles, and the multiplicative repair charges a whole row for its single worst channel. So the 3% gap above is agreement, not a discrepancy – and every future comparison of two CHORD D values has to be read against this floor.

The ladder is on L , and it is a set of upper bounds rather than a curve. The obvious ladder – scaling N_{freq} down by powers of two – is the wrong one: with constraint generation on, per-LP cost is *flat* in N_{freq} , so every rung stays a full-price 409 600-LP job, and each rung needs its own GPU sweep. Scaling the coarse-graining level instead gives $N_\beta(L) = 2^{16-L} \times 400$, so each increment of L halves the LP count exactly, every rung is `coarse_grain(L)` of one file with no GPU time at all, and the top rung *is* the production run. Forty cells were measured across rungs 0–8 at $K = 16, 32, 64, 128$; at $K = 128$:

²Campaign 3’s cost figures are extrapolations from subsampled cells, carrying that campaign’s own $\pm 27\%$ spread, whereas campaign 4’s are measured full runs; the two should not be compared as equals. Campaign 3 also quotes 414 208 covering LPs for this cell against the 409 600 measured here.

rung	0	1	2	3	4	5	6	7	8	prod.
L	16	15	14	13	12	11	10	9	8	6
N_β	400	800	1600	3200	6400	12800	25600	51200	102400	409 600
D	0.1611	0.0870	0.0519	0.0350	0.0289	0.0276	0.0276	0.0258	0.0250	0.0243
D/D_{prod}	6.63	3.58	2.13	1.44	1.19	1.14	1.14	1.06	1.03	1

A power law fitted up the ladder is wrong – it lands at $0.48\times$ the true rung-8 value over rungs 0–8 – because the low rungs *saturate in K* : at rung 0 an eightfold increase in rank buys only $1.35\times$ in D , against $5.67\times$ at production, since there the error is set by the max-envelope and not by the rank. The approach is not even monotone (rung 6 sits 0.1% above rung 5, comfortably inside the 6% floor). **But every rung above its own knee is a $1.03\text{--}1.07\times$ conservative estimate of the production D** , which is the useful way to spend the ladder: iterate at rung 5 (19.9 core-hours, $1.14\times$) or rung 7 (79.5 core-hours, $1.06\times$), never at rung 2 ($2.13\times$).

The W -step is closed. A production W -step costs **6202 core-hours** – $12.9\times$ **the entire Q -step** – and would buy $\approx 17\%$ in D , which is $9\times$ worse per core-hour than a rank doubling *even if it drove D to zero*. The cause is specific: constraint generation does not run at all in the W direction, because none of the sampled channels has a bounded first relaxation, so every LP falls back to the exact 409 600-row solve at 793 core-s each. **The production recipe is Q -only**, and this should not be revisited without a reason.

The way this was found is worth recording. A subscale study at $N_\beta \leq 1600$ concluded the opposite – that a W -step was $1.8\times$ *better* per core-hour than buying rank – and projected the production cost at $2\text{--}13\%$ of a Q -step. It missed by $97\text{--}564\times$. The probe was careful and calibrated to $0.988\times$; the flaw was that its N_β -independent term is only independent *while the cut loop runs*, and at production the loop never runs. **A cost model fitted where an optimisation works cannot see the cliff where that optimisation stops working** – and the same shape appears above for D , where the low rungs cannot see what the production cell spends its rank on.

Three corrections to the shipped code.

- **cuts_pool_window should be 4, not 0.** At $K = 128$ this is the difference between 1026 and 480 core-hours for a 10^{-14} change in the answer. It is a *high-rank* knob, not a scale knob: $2.15\times$ at $K = 128$, $1.04\times$ at $K = 64$, nothing below. Both the old code and the port ship the losing value, and both say in their own comments that it is the losing value.
- **The repair pipeline is worth 0.087% at CHORD**, not the $2.5\times$ its docstring claims from small geometries.
- **One correctness bug, and the port introduced it.** At the shipped `for.wstep()` default, `wstep()` can return a map that does not dominate the reference while reporting `is_admissible = True` and a finite D – precisely the failure the one-sided distance exists to prevent. The additive repair decides whether to act from the *ratio*, which cannot see a product entry that is negative where the reference is positive, while the lift it computes is driven by the *deficit*, which can. The old `free_lp.py` gates on the deficit and is correct. `recommended('w')` is safe, so no campaign-4 measurement is affected, but the defect grows with scale: 120 negative entries at production against 1 at rung 2. The bit-identity gates do not cover the case, which is a gap in the gates as much as a bug in the code.

Should the old code be deleted? Yes, with one exception. No campaign-4 agent imported any old code – verified by `grep` across nine working directories, zero imports – and no published number failed to reproduce. The `slow_avar` modules can go once four self-test calls are removed from `__main__.py` and `varmap/tests.py`'s cross-check helper is made to tolerate the missing import; its skip path already exists.

The exception is `research/toplevel/*.py`. It is untracked scratch, so deleting it buys no tidiness, and it is currently the only oracle that establishes what the additive repair’s correct behaviour is. Fix the bug above against it, add a gate case with a negative product entry at ratio ≤ 1 , and then delete it.

The benchmark itself – every cell, its recipe, its machine conditions and its solver counts – is in `research/toplevel/BENCHMARK` with a machine-readable copy in `benchmark_cells.json`. **Rank beyond 128 is the only arm campaign 4 tested that beat everything it was compared against**, and the ladder now makes it cheap to ask what $K = 256$ buys before paying for it at production.

A Variance maps of a config's trees are row-restrictions of one another

Label the trees of one `DedispersionConfig` by a pair (γ, e) : primary-tree index γ (downsampling factor 2^γ) and early-trigger level e . Each tree has its own variance map $A^{(\gamma, e)}$, a $(D_{\gamma, e} M_{\gamma, e} P_\gamma)$ -by- N_{freq} matrix. Say that A' is a **row-restriction** of A if there is an injection from the rows of A' to the rows of A under which corresponding rows are equal as vectors in $\mathbb{R}^{N_{\text{freq}}}$. We prove:

1. $A^{(\gamma, e)}$ is a row-restriction of $A^{(\gamma, 0)}$, for every γ and e .
2. In the “no-detrender” case (i.e. if the linear operator L does not include a `GpuDetrenderLps2d`), $A^{(\gamma, 0)}$ is a row-restriction of $A^{(0, 0)}$, for every γ .

In both cases the injection is explicit and is given below, which is what makes the result usable rather than merely existential.

Scope. The two propositions need different hypotheses, and it is worth separating them. Proposition 1 holds whatever the upstream processing, and in particular for L exactly as defined in §1.1, `GpuDetrenderLps2d` included: the two trees it compares have the same γ , so they are fed by one and the same upstream chain (*dedispersion.pdf*, “Time-downsampled trees and early triggers”), and the proof shows that their operators agree row by row – which needs nothing about what produced their common input, nor about the statistics of that input. Proposition 2 compares trees fed by *differently* downsampled inputs, and is proved only in the no-detrender setting of §D.1 (tree gridding $\rightarrow$ subbanded tree dedispersion $\rightarrow$ peak-finding convolution).

A.1 Inputs

Write r_{top} for the config's toplevel tree rank and R for the `pf_rank` of its toplevel subband counts. We use the following facts about how a config's trees are built (*dedispersion.pdf*, “Time-downsampled trees and early triggers”, *dedispersion.pdf*, “Subband search”).

(F1) Ranks. Tree (γ, e) performs rank- $\hat{r}$ dedispersion, $\hat{r} \equiv r_{\text{top}} - e$, on the lowest $2^{\hat{r}}$ tree-freq channels of its input array, and its `pf_rank` is $R_e = R - e$ (an early trigger removes one subband level per level of trigger). Its exported coarse-DM index runs over $0 \leq d < D_{\gamma, e} = 2^{\text{tree_r} - R_e}$, where $\text{tree_r} = \hat{r}$ for $\gamma = 0$ and $\hat{r} - 1$ for $\gamma > 0$; the trees with $\gamma > 0$ keep only the upper half of the coarse DM range.

(F2) Subband sets. Let $\mathcal{B}(\gamma, e)$ be the set of *toplevel* tree-freq ranges $[f_{\text{lo}}, f_{\text{hi}}]$ searched by tree (γ, e) – in the code, the set of pairs $(\text{n_to_toplevel_flo}(n), \text{n_to_toplevel_fhi}(n))$ over its subbands n . Then

$$\mathcal{B}(\gamma, e) \subseteq \mathcal{B}(\gamma, 0) = \mathcal{B}(0, 0) \quad \text{for every } (\gamma, e). \quad (\text{A.1})$$

That is: *time downsampling does not change which bands are searched, and an early trigger may only remove bands, never add or alter one.* This is a property of the config-to-tree construction and is unit-tested directly (`test_subband_property` in `pirate_frb/tests/test_subbands.py`, run by `python -m pirate_frb test --sb`), over random configs and every shipped config; we take it as given here.

(F3) Locality of a subband output. Let X be the array that tree (γ, e) dedisperses – whatever the upstream chain produced – and let $b \in \mathcal{B}(\gamma, e)$ be a band at level l , occupying the 2^K tree-freq channels $a \leq f < a + 2^K$ with $K \equiv \hat{r} - R_e + l$. Then, indexing the tree's multiplets as $m = (b, j)$ with $0 \leq j < 2^l$ (*dedispersion.pdf*, “Subband search”), the tree's output at trial $(d, (b, j), p)$ is

$$h_p * \left[\text{DD}(K) X|_{[a, a+2^K)} \right]_{2^l d + j + \sigma_\gamma 2^{K-1}} \quad (\text{up to a time shift}), \quad \sigma_\gamma \equiv \begin{cases} 0 & \gamma = 0 \\ 1 & \gamma > 0, \end{cases} \quad (\text{A.2})$$

where $\text{DD}(K)$ is the rank- K dedispersion transform and h_p the peak-finding kernel. In particular the output depends on X only through the 2^K channels of b , and the recipe – which channels, which rank- K DM index, which profile – does not refer to the ambient tree. This is the content of the first two paragraphs of the proof of Lemma 1 in §B (its Cases 1 and 2, and the identification of the rank- K DM index with $2^l d_{\text{hi}} + d_{\text{lo}}$), together with the upper-half restriction of (F1); it is algebra about the dedispersion transform and uses nothing about X . The time shift is the subband extraction lag, which drops out of everything below.

(F4) Closed form (no detrender only). Lemma 1 of §B – whose proof is independent of this appendix, so there is no circularity – evaluates (A.2) in the setting of §D.1. With b, l, a, K as in (F3) and an input channel F , put

$$\Phi_{b,p,F}(x) \equiv 2^{-K} \sum_{u=0}^{2^K-1} \sum_{v=0}^{2^K-1} w_{(a+u)F} w_{(a+v)F} A_p(L_K(u, x) - L_K(v, x)), \quad (\text{A.3})$$

where $w_{fF} \geq 0$ are the tree-gridding weights, L_K is the lag function and A_p is the autocorrelation of h_p . Then

$$A_{(d, (b,j), p), F}^{(\gamma, e)} = \Phi_{b,p,F}(2^l d + j + \sigma_\gamma 2^{K-1}). \quad (\text{A.4})$$

(F5) Profiles and max.width. The number of peak-finding profiles is $P = 1 + 3 \log_2 \text{max.width}$, where `max.width` is the widest boxcar the peak-finder searches, measured in the tree’s own time samples. Profile p denotes the same filter h_p whatever `max.width` is: $p = 0$ is the identity, and $p = 3l + q$ with $q \in \{1, 2, 3\}$ is one of three shapes built from length- 2^l boxcars, so `max.width` enters only through how many levels l exist. The bank for `max.width` is therefore a *prefix* of the bank for 2max.width : doubling the width appends one level, i.e. three profiles, at the end. Now, `max.width` is a per-*primary*-tree config field, which the `DedispersionPlan` constructor copies verbatim onto each tree, so every tree of one family reads the same value; and `DedispersionConfig::validate()` requires each downsampled primary tree’s `max.width` to equal its predecessor’s or be half of it. Hence

$$P_\gamma = 1 + 3 \log_2 \text{max.width}_\gamma \quad \text{is independent of } e, \text{ and non-increasing in } \gamma. \quad (\text{A.5})$$

A.2 Three consequences of (F1), (F2) and (F5)

Observation. *For every tree of the config,*

$$\hat{r} - R_e = (r_{\text{top}} - e) - (R - e) = r_{\text{top}} - R, \quad (\text{A.6})$$

independent of (γ, e) . Consequently:

- (a) *One coarse-freq channel spans $2^{r_{\text{top}}-R}$ toplevel tree-freq channels in every tree. Hence a band $b \in \mathcal{B}(0, 0)$ occupies $2^l 2^{r_{\text{top}}-R}$ toplevel channels in whichever tree searches it, so its level l , its multiplet count 2^l , its first channel a , and therefore $K = r_{\text{top}} - R + l$, are attributes of b alone and do not depend on which tree we are in.*
- (b) *$D_{\gamma, e}$ does not depend on e : $D_{\gamma, e} = D_\gamma$ with $D_0 = 2^{r_{\text{top}}-R}$ and $D_\gamma = 2^{r_{\text{top}}-R-1}$ for $\gamma > 0$.*

Proof. (A.6) is immediate from (F1). For (a), a level- l band spans 2^l coarse-freq channels by definition, and by (A.6) a coarse-freq channel is $2^{\hat{r}-R_e} = 2^{r_{\text{top}}-R}$ toplevel channels wide in every tree; since a tree’s tree-freq channels are literally the lowest toplevel ones (F1), a is the same integer read as a tree-freq or a toplevel index. The width therefore determines l , and b determines (l, a) , in a tree-independent way. For (b), substitute (A.6) into $D_{\gamma, e} = 2^{\text{tree.r}-R_e}$, using `tree.r` - $R_e = \hat{r} - R_e$ for $\gamma = 0$ and $\hat{r} - R_e - 1$ for $\gamma > 0$. $\square$

Part (b) is the fact that makes statement 1 a restriction in the *multiplet* index only: an early trigger does not shorten the coarse-DM axis.

(c) Profile ranges nest. P is an attribute of the primary-tree family, so we may write P_γ without an e ; and $P_\gamma \leq P_0$ for every γ , with equality iff the two families share a `max_width`. Since profile p denotes the same filter in every tree, the profile index set $[0, P_\gamma)$ of any tree is contained in that of tree $(0, 0)$, and an index the two trees share means the same thing in both.

Proof. Immediate from (F5): all trees of family γ read one `max_width`, and (A.5) is non-increasing in γ . $\square$

Part (c) is what makes statement 2 well-formed: without it, a downsampled tree could search profiles that tree $(0, 0)$ does not, and the right-hand side of (A.8) would be indexed out of range.

A.3 Statement 1: early triggers

Proposition 1. Fix γ and $e \geq 0$, and let the trees (γ, e) and $(\gamma, 0)$ be fed by a common upstream chain, which may be arbitrary. Then for every band $b \in \mathcal{B}(\gamma, e)$ at level l , every $0 \leq j < 2^l$, every $0 \leq d < D_\gamma$, every $0 \leq p < P_\gamma$ and every F ,

$$A_{(d, (b, j), p), F}^{(\gamma, e)} = A_{(d, (b, j), p), F}^{(\gamma, 0)}. \quad (\text{A.7})$$

Hence $A^{(\gamma, e)}$ is a row-restriction of $A^{(\gamma, 0)}$: it is obtained by deleting the rows whose band lies in $\mathcal{B}(\gamma, 0) \setminus \mathcal{B}(\gamma, e)$, with (d, j, p) carried over unchanged.

Proof. We show the two trees' linear operators agree row by row; the statement about A then follows, since $A_{\alpha F} = \sum_{t'} L[\alpha t, Ft']^2$ (and, for $\gamma > 0$, a sum over the 2^γ polyphase components, which is the same set for both trees because γ is).

By (F2), $b \in \mathcal{B}(\gamma, e) \subseteq \mathcal{B}(\gamma, 0)$, so both sides of (A.7) are defined. By the Observation (b) both trees have the same coarse-DM range $0 \leq d < D_\gamma$, and by (a) the band contributes multiplets (b, j) , $0 \leq j < 2^l$, with the same l , the same a and the same K in both. Both carry the same σ_γ , since γ is the same on both sides. Having the same γ , the two trees dedisperse the same array X – tree (γ, e) using only its lowest $2^{\hat{r}}$ channels, which do contain b , since $a + 2^K \leq 2^{\hat{r}}$ whenever tree (γ, e) searches b . So (F3) applies to each side and returns the same expression (A.2) in X . $\square$

Note what the proof does *not* need: no relation between the two trees' DM grids has to be established, because by (b) they are the same grid; and nothing at all is assumed about X , which is why the detrended operator of §1.1 is covered. Early triggers are a pure restriction in m .

A.4 Statement 2: time downsampling

Proposition 2. Work in the no-detrender setting of §D.1, and fix $\gamma > 0$. For every band $b \in \mathcal{B}(0, 0)$ at level l , every $0 \leq j < 2^l$, every $0 \leq d < D_\gamma$, every $0 \leq p < P_\gamma$ and every F ,

$$A_{(d, (b, j), p), F}^{(\gamma, 0)} = A_{(d+D_0-D_\gamma, (b, j), p), F}^{(0, 0)}, \quad D_0 - D_\gamma = \frac{1}{2}D_0. \quad (\text{A.8})$$

Hence $A^{(\gamma, 0)}$ is a row-restriction of $A^{(0, 0)}$: it is the block of rows whose coarse-DM index lies in the upper half $D_0/2 \leq d < D_0$, with (m, p) carried over unchanged.

Proof. By (F2) with $e = 0$, the two trees search the same band set $\mathcal{B}(\gamma, 0) = \mathcal{B}(0, 0)$, so by the Observation (a) they have the same multiplet set and the same l , a , K , hence the same $\Phi_{b, p, F}$ of (A.3); in particular $M_{\gamma, 0} = M_{0, 0}$. By the Observation (b), $D_\gamma = D_0/2$, so $D_0 - D_\gamma = D_0/2$ and the map $d \mapsto d + D_0/2$ is an injection $[0, D_\gamma) \rightarrow [0, D_0)$.

Apply (A.4) to each side. The left-hand side has $\gamma > 0$, hence $\sigma_\gamma = 1$:

$$A_{(d, (b, j), p), F}^{(\gamma, 0)} = \Phi_{b, p, F}(2^l d + j + 2^{K-1}). \quad (\text{A.9})$$

The right-hand side has $\gamma = 0$, hence $\sigma_0 = 0$, and coarse-DM index $d + D_0/2$:

$$A_{(d+D_0/2, (b,j), p), F}^{(0,0)} = \Phi_{b,p,F}(2^l(d + D_0/2) + j) = \Phi_{b,p,F}(2^l d + j + 2^l D_0/2). \quad (\text{A.10})$$

Finally $2^l D_0/2 = 2^l 2^{r_{\text{top}}-R-1} = 2^{(r_{\text{top}}-R+l)-1} = 2^{K-1}$ by the Observation (a), so the two arguments of $\Phi_{b,p,F}$ agree. $\square$

A word on the profile index. The right-hand side of (A.8) is defined because $P_\gamma \leq P_0$ by the Observation (c), and a profile index the two trees share denotes the same filter by (F5) – which is why p can be “carried over unchanged” between trees that do not have the same number of profiles. So Proposition 2’s restriction is in the coarse-DM index *and* the profile index, whereas Proposition 1’s is in the multiplet index alone. The profile restriction is not vacuous: P_γ is often strictly less than P_0 . In `configs/dedispersion/chord_sb2_et.yml`, whose four primary trees have `max_width` = 32, 16, 8, 4, the profile counts are $P = 16, 13, 10, 7$. Anything deriving $A^{(\gamma,0)}$ from $A^{(0,0)}$ must therefore select the rows with $p < P_\gamma$ rather than assume the profile axes match – the array shapes look plausible either way.

Two remarks. First, in the no-detrender setting (A.4) depends on γ only through $\sigma_\gamma \in \{0, 1\}$ – the down-sampling factor 2^γ itself never appears. So any two trees of a config with $\gamma, \gamma' > 0$ and the same e have *identical* rows at every profile index both of them have, i.e. for $0 \leq p < \min(P_\gamma, P_{\gamma'})$. Their maps coincide outright when the two families share a `max_width`, and are nested otherwise: in `chord_sb2_et.yml` the three downsampled families have $P = 13, 10, 7$, so there they are nested. Second, both propositions have been checked numerically over random configs, against maps computed in float64 by the algorithm of §D: the row identities (A.7) and (A.8) hold to 0 and one ulp respectively. Proposition 1 has additionally been checked with a `GpuDetrenderLps2d` in front of the dedisperser, where it continues to hold and Proposition 2 fails outright.

B Monotonicity of the variance map in the DM bits (no detrender)

This appendix treats the no-detrender case only. The linear operator is bare tree gridding $\rightarrow$ sub-banded tree dedispersion $\rightarrow$ peak-finding convolution, as in §D, §D.1 – *not* the detrended operator of §1.1, which includes a `GpuDetrenderLps2d`. **The result below is proved in that case only, and the restriction is not a technicality:** with a `GpuDetrenderLps2d` in front of the dedisperser the property is not merely unproved but *false*, and badly so (§B.5).

Fix one column of the variance map: a `DedispersionTree`, a frequency subband, a peak-finding profile p , and a single input frequency channel F . What is left is a 1-d array $A[d_m]$ indexed by the subband’s dispersion measure $d_m = 2^l d_{\text{hi}} + d_{\text{lo}}$ (*dedispersion.pdf*, “Subband search”). The purpose of this appendix is to prove that A is *antitone in the binary digits of d_m* : writing $d \subseteq d'$ for the bitwise-subset partial order on DM indices (i.e. $d_b \leq d'_b$ for every bit b , equivalently $d \& d' = d$),

$$d \subseteq d' \implies A[d'] \leq A[d]. \quad (\text{B.1})$$

Equivalently: *setting a bit of the DM index never increases the variance*, $A[d \mid 2^b] \leq A[d]$. Two consequences are worth stating separately. First, taking $d = 0$, the DM index $d_m = 0$ maximizes the column. Second, and more useful, the *dyadic-block property*: in every aligned dyadic block of d_m indices, the *first* entry is the largest,

$$A[2^k j] = \max_{0 \leq i < 2^k} A[2^k j + i] \quad \text{for every } k \geq 0, j \geq 0 \text{ with indices in range,} \quad (\text{B.2})$$

which follows from (B.1) because $2^k j$ is the bitwise-minimal element of its own dyadic block ($2^k j$ and $i < 2^k$ have disjoint bits). The practical consequence is that an approximation to the variance map which resolves only the top DM bits, and which must never *underestimate* a variance (§1.2), can simply be evaluated at

the bottom of each dyadic DM block: no maximization over the block is needed, and the resulting bound is tight.

Concretely, A here is a reindexing of the variance map of Eq. (1.6): with $m = (\text{subband}, d_{\text{lo}})$ and $d = d_{\text{hi}}$ as in §D.5, the per-multiplet variance $A_{\alpha F}$ at $\alpha = (d_{\text{hi}}, m, p)$ is $A[2^l d_{\text{hi}} + d_{\text{lo}}]$, so a dyadic block of A generally runs across several multiplets.

Note that A is *not* monotone in d_m as an integer: it wanders up and down, but never above the value at any bitwise-subset index (§B.5).

The proof splits into a statement about the lag function $L_r(f, d)$ (Lemmas 2–5: *the lag function is linear in the bits of d with non-negative coefficients, so setting a DM bit can only spread the impulse response out*) and a statement about the peak-finding kernels (Lemma 6: *spreading out can only decrease the variance*). Both are needed; §B.5 shows that neither hypothesis can be dropped, and that the bitwise-subset order is exactly the right partial order. **The first hypothesis – non-negativity of the gridding weights – is exactly what a detrender destroys, which is why the no-detrender assumption above cannot be relaxed.**

B.1 Setup

We work in the no-detrender setting of §D.1: the linear operator is (tree gridding) $\rightarrow$ (subbanded tree dedispersion) $\rightarrow$ (convolution with the peak-finding profile h_p), with no normalization to sigmas and no coarse-graining, and the dedisperser input has uncorrelated entries with per-channel variance V_F .

Fix a `DedispersionTree` with early-trigger level e and primary-tree index γ , and write

$$\hat{r} \equiv r_{\text{top}} - e \quad (\text{B.3})$$

for the rank of the dedispersion it performs: the tree dedisperses the lowest $2^{\hat{r}}$ tree-freq channels of the band (*dedispersion.pdf*, “Time-downsampled trees and early triggers”). (In the code, `tree_r` is $\hat{r}$ for $\gamma = 0$ and $\hat{r} - 1$ for $\gamma > 0$, because the upper-DM-half restriction is folded into the rank.) Fix a subband (l, s) , $0 \leq l \leq R$, and let

$$K \equiv \hat{r} - R + l, \quad a \equiv 2^{\hat{r}-R} \times (\text{index of the subband's first coarse-freq block}), \quad (\text{B.4})$$

so that the subband occupies the 2^K contiguous tree-freq channels $a \leq f < a + 2^K$ (*dedispersion.pdf*, “Subband search”) and its DM index runs over $0 \leq d_m < 2^K$. Finally let

$$w_u \equiv w_{(a+u)F} \geq 0 \quad (0 \leq u < 2^K) \quad (\text{B.5})$$

be the tree-gridding weights (*dedispersion.pdf*, “Tree gridding kernel”) of input channel F on the subband’s channels. Non-negativity is the only property of them we use (they are overlap lengths; that their support is a short contiguous run is not needed).

Lemma 1 (closed form). *For a $\gamma = 0$ tree, the variance-map column is*

$$A[d_m] = 2^{-K} \sum_{u=0}^{2^K-1} \sum_{v=0}^{2^K-1} w_u w_v A_p(L_K(u, d_m) - L_K(v, d_m)), \quad (\text{B.6})$$

where L_K is the lag function of *dedispersion.pdf*, “Non-recursive formulation of tree dedispersion” and A_p is the autocorrelation of the peak-finding kernel h_p . For a $\gamma > 0$ tree, the exported array is the upper DM half of (B.6): $A[d_m] = (\text{RHS evaluated at } d_m + 2^{K-1})$.

Proof. Both cases of subbanded dedispersion (*dedispersion.pdf*, “Subband search”) compute exactly $\text{DD}(K)$ applied to the subband’s 2^K tree-freq channels, followed by a time shift. In Case 1 ($l = 0$ or even s) the subband output is read off the intermediate array after $K = \hat{r} - R + l$ dedispersion steps, whose coarse-freq

block f is $\text{DD}(K)$ applied to tree-freq channels $2^K f \leq \cdot < 2^K (f+1)$ (Observation 1 of *dedispersion.pdf*, “The non-recursive formulation of tree dedispersion”). In Case 2 ($l > 0$, odd s) the two adjacent level- $(K-1)$ blocks $2f+1$ and $2f+2$ are combined by one further DDS step, in which block $2f+1$ plays the “lower” role and $2f+2$ the “upper” role. This pair is not aligned in the ambient tree, but the DDS formulas do not depend on the block index (Observation 1), so the last-step identity $\text{DD}(K) = \text{DDS}(K-1) \circ (1 \otimes \text{DD}(K-1))$ applies to the pair verbatim: the result is $\text{DD}(K)$ applied to their union, which is precisely the subband.

In both cases the rank- K DM index of that $\text{DD}(K)$ is the subband DM index $d_m = 2^l d_{\text{hi}} + d_{\text{lo}}$. In Case 1 this is immediate ($d = d_{\text{hi}} 2^l + d_{\text{lo}}$ in the notation of *dedispersion.pdf*, “Subband search”). In Case 2 the extra DDS step maps the level- $(K-1)$ index $d = d_{\text{hi}} 2^{l-1} + d_{\text{lo}}$ to $2d+b$, i.e. to $2^l d_{\text{hi}} + (2d_{\text{lo}} + b)$, and $2d_{\text{lo}} + b$ is exactly the fine index of the multiplet which receives it.

Therefore, by the non-recursive form of $\text{DD}(K)$ (*dedispersion.pdf*, “Non-recursive formulation of tree dedispersion”) and the one-hot impulse response (D.5), the impulse response of input channel F in this subband is

$$G_{F,d_m}[t] = 2^{-K/2} \sum_u w_u \delta_{t, L_K(u, d_m) + T}, \quad (\text{B.7})$$

and Eqs. (D.3), (D.7) give (B.6), per unit input variance V_F . The subband extraction lag T drops out because $\|h_p * g\|^2$ is translation invariant; note that this holds for *any* T , so the variance map is unchanged by the alternative time-lag convention discussed in the Remark of *dedispersion.pdf*, “Subband search”.

For $\gamma > 0$, the tree time-downsamples its input by 2^γ before dedispersing. Downsampling adds 2^γ samples and multiplies by $2^{-\gamma/2}$, so the downsampled array again has uncorrelated entries with per-channel variance V_F , and the argument above applies verbatim on the downsampled time grid. The only other difference is that the tree keeps just the upper half of the coarse DM range (*dedispersion.pdf*, “Time-downsampled trees and early triggers”): the top bit of d_{hi} is fixed to 1 and dropped, which is the DM range $d_m \geq 2^{K-1}$. $\square$

Two remarks. First, early triggers have disappeared into the definition of $\hat{r}$: a tree with $e > 0$ performs ordinary rank- $\hat{r}$ dedispersion on the first $2^{\hat{r}}$ tree-freq channels, i.e. the same statement with a truncated channel map. Second, only $w_u \geq 0$ is used below, so everything that follows holds for an arbitrary channel map and an arbitrary set of subband counts C_l , and – summing (B.6) against $V_F \geq 0$ – for any non-negative combination of input channels.

B.2 Setting a DM bit spreads the impulse response out

Lemma 2 (bitwise form of L). *Write d_m for bit m of d . Then*

$$L_r(f, d) = \sum_{m=0}^{r-1} (1 - f_{r-1-m}) c^{(m)}(d), \quad c^{(m)}(d) \equiv \left\lfloor \frac{d}{2^{m+1}} \right\rfloor + d_m \geq 0, \quad (\text{B.8})$$

and for every $0 \leq m_0 < r$ (with $0 \leq d < 2^r$),

$$\sum_{m=m_0+1}^{r-1} c^{(m)}(d) = \left\lfloor \frac{d}{2^{m_0+1}} \right\rfloor = c^{(m_0)}(d) - d_{m_0}. \quad (\text{B.9})$$

In particular $L_r(0, d) = d$ and $L_r(2^r - 1, d) = 0$.

Proof. In the definition of the lag function $L_r(f, d)$ (*dedispersion.pdf*, “Non-recursive formulation of tree dedispersion”), the coefficient of $(1 - f_i)$ is

$$\sum_{j=0}^{r-1} \Omega_{i+j-r} d_j = \Omega_{-1} d_{r-1-i} + \sum_{j \geq r-i} 2^{i+j-r} d_j = d_{r-1-i} + 2^{i-r} \cdot 2^{r-i} \left\lfloor \frac{d}{2^{r-i}} \right\rfloor = d_m + \left\lfloor \frac{d}{2^{m+1}} \right\rfloor \quad (\text{B.10})$$

with $m \equiv r - 1 - i$, which is (B.8). For (B.9), use $\lfloor d/2^m \rfloor = 2\lfloor d/2^{m+1} \rfloor + d_m$ to rewrite $c^{(m)}(d) =$

$\lfloor d/2^m \rfloor - \lfloor d/2^{m+1} \rfloor$; the sum telescopes to $\lfloor d/2^{m_0+1} \rfloor - \lfloor d/2^r \rfloor$, and $\lfloor d/2^r \rfloor = 0$. Telescoping from $m = 0$ instead gives $L_r(0, d) = \sum_m c^{(m)}(d) = d$, while $f = 2^r - 1$ kills every term. $\square$

Lemma 3 (monotonicity in frequency). *For fixed d , the lag $L_r(f, d)$ is non-increasing in f . More precisely, if $f < f'$ and i_0 is the highest bit in which they differ, then*

$$L_r(f, d) - L_r(f', d) \geq d_{r-1-i_0} \geq 0. \quad (\text{B.11})$$

Proof. Write $m_0 = r - 1 - i_0$; since $f < f'$ we have $f_{i_0} = 0$ and $f'_{i_0} = 1$. In the difference of two copies of (B.8), terms with $i > i_0$ cancel, the $i = i_0$ term contributes $+c^{(m_0)}(d)$, and each term with $i < i_0$ (i.e. $m > m_0$) contributes at least $-c^{(m)}(d)$. Hence

$$L_r(f, d) - L_r(f', d) \geq c^{(m_0)}(d) - \sum_{m > m_0} c^{(m)}(d) = d_{m_0} \quad (\text{B.12})$$

by (B.9). $\square$

Lemma 3 is the algebraic form of an obvious physical statement: tree dedispersion delays the high tree-freq channels less than the low ones, the total spread being $L_r(0, d) - L_r(2^r - 1, d) = d$. Note that the lag difference can vanish even when $f \neq f'$ (when $d_{m_0} = 0$ and the lower bits conspire); this is the “relative lags happen to cancel” phenomenon which makes **SparseTile**’s content bits sometimes a proper subset of the top $(i_0 + 1)$ bits (§D.3).

Lemma 4 (the lag function is linear in the DM bits). *For fixed f , the lag function is a non-negative linear form in the binary digits of d :*

$$L_r(f, d) = \sum_{b=0}^{r-1} d_b \lambda_b(f), \quad \lambda_b(f) \equiv L_r(f, 2^b) = (1 - f_{r-1-b}) + \sum_{m < b} (1 - f_{r-1-m}) 2^{b-1-m} \geq 0. \quad (\text{B.13})$$

Equivalently, $L_r(f, \cdot)$ is additive on bitwise-disjoint DM indices: if $d \& e = 0$ then $L_r(f, d|e) = L_r(f, d) + L_r(f, e)$.

Proof. Expanding $\lfloor d/2^{m+1} \rfloor$ in binary, $c^{(m)}(d) = \lfloor d/2^{m+1} \rfloor + d_m = d_m + \sum_{j > m} 2^{j-m-1} d_j$, which is a linear form in the bits of d with non-negative integer coefficients; substituting into (B.8) and exchanging the order of summation gives (B.13), and evaluating it at $d = 2^b$ identifies $\lambda_b(f)$. A linear form in the bits is additive whenever the bits add, i.e. whenever the two arguments are bitwise disjoint. $\square$

Physically: each DM *bit* contributes its own delay ramp $\lambda_b(\cdot)$ across the band, and a general DM index simply superposes the ramps of its set bits. (The lag is *not* linear in d itself: $\lambda_b(f) \neq 2^b \lambda_0(f)$ in general, the difference being the rounding built into $\Omega_{-1} = 1$.)

Lemma 5 (bit spreading). *Write $\Delta_{uv}(d) \equiv L_K(u, d) - L_K(v, d)$. Then for all $u \leq v$ and all bitwise-disjoint d, e ,*

$$\Delta_{uv}(d|e) = \Delta_{uv}(d) + \Delta_{uv}(e) \geq \Delta_{uv}(d) \geq 0. \quad (\text{B.14})$$

Consequently $d \subseteq d'$ implies $\Delta_{uv}(d') \geq \Delta_{uv}(d) \geq 0$: setting DM bits can only increase every pairwise separation of the impulse response (B.7).

Proof. The equality is Lemma 4 applied to u and to v and subtracted. Both inequalities are Lemma 3: $\Delta_{uv}(x) \geq 0$ for $u \leq v$ and any x , applied to $x = e$ and to $x = d$. For the consequence, take $e = d' \setminus d$ (bitwise), so that $d' = d|e$ with $d \& e = 0$. $\square$

Three remarks.

First, *when is the increment zero?* By (B.8), $\Delta_{uv}(2^b) = \sum_{m \leq b} \alpha_m c^{(m)}(2^b)$ with $\alpha_m = v_{K-1-m} - u_{K-1-m}$, since $c^{(m)}(2^b) = 0$ for $m > b$. If i_0 is the highest bit in which u and v differ and $m_0 = K - 1 - i_0$, then

$\alpha_m = 0$ for $m < m_0$, so $\Delta_{uv}(2^b) = 0$ whenever $b < m_0$, while $\Delta_{uv}(2^{m_0}) \geq 1$ by (B.11). (For $b > m_0$ the increment can also vanish accidentally, by the cancellation noted after Lemma 3.) So a low DM bit b is felt only by pairs of channels lying in different coarse-frequency blocks of size 2^{K-b} ; a pair of nearby channels sees no change at all. This is why, numerically, most instances of (B.1) are exact equalities, increasingly so for the low DM bits: the gridding weights w_u of one input channel are supported on a short contiguous run of tree-freq channels.

Second, Lemma 5 contains the “dyadic” statement that within an aligned dyadic block of DM indices every pairwise separation is smallest at the first index of the block: that is the case $(d, e) = (2^k j, i)$ with $0 \leq i < 2^k$.

Third, the same dyadic statement can be read off the lag-additivity identity of *dedispersion.pdf*, “The splitting identity” with $r'' = k$, $r' = K - k$: writing $u = 2^{K-k}\bar{u} + u'$,

$$L_K(u, 2^k j + i) = L_{K-k}(u', j) + (2^k - 1 - \bar{u})j + L_k(\bar{u}, i), \quad (\text{B.15})$$

whose only i -dependent term $L_k(\bar{u}, i)$ depends on u only through its coarse block index $\bar{u}$, and is non-increasing in $\bar{u}$ by Lemma 3. In words: the low DM bits perform a rank- k dedispersion of the coarse blocks, sliding them apart while leaving the arrangement of channels within each block untouched. This is a useful picture, but it is not needed for the proof: Lemma 4 supersedes it.

B.3 Spreading out decreases the variance

Lemma 6 (kernel autocorrelations are unimodal). *For every peak-finding profile p (*dedispersion.pdf*, “Peak-finding”), the autocorrelation $A_p[\delta] = \sum_t h_p[t]h_p[t+\delta]$ is even in δ and non-increasing in $|\delta|$.*

Proof. Evenness is immediate. Write B_n for the boxcar of length n (all entries 1). Each peak-finding kernel is a non-negative combination of boxcars which share a common center and whose lengths have a common parity: with $w = 2^\lambda$,

$$h_{\lambda,0} = B_w, \quad h_{\lambda,1} = B_{2w}, \quad h_{\lambda,2} = \tfrac{1}{2}B_{3w} + \tfrac{1}{2}B_w, \quad h_{\lambda,3} = \tfrac{1}{2}B_{4w} + \tfrac{1}{2}B_{2w}, \quad (\text{B.16})$$

where in each case the boxcars are centered on the center of $h_{\lambda,q}$ (e.g. $[\frac{1}{2}, 1, \frac{1}{2}] = \frac{1}{2}[1, 1, 1] + \frac{1}{2}[0, 1, 0]$). Hence A_p is a non-negative combination of cross-correlations $\phi_{mn}[\delta] = \sum_t B_m[t]B_n[t+\delta]$ of co-centered boxcars, and for $m \equiv n \pmod{2}$ and integer δ ,

$$\phi_{mn}[\delta] = \max\left(0, \min\left(m, n, \frac{m+n}{2} - |\delta|\right)\right), \quad (\text{B.17})$$

which is even and non-increasing in $|\delta|$. A non-negative combination of such functions has the same property. $\square$

B.4 The theorem

Theorem 7. *Let $K \geq 0$, let $w_u \geq 0$ for $0 \leq u < 2^K$, and let $\phi : \mathbb{Z} \rightarrow \mathbb{R}$ be even and non-increasing on $\delta \geq 0$. Define*

$$A[d] = \sum_{u,v} w_u w_v \phi(L_K(u, d) - L_K(v, d)), \quad 0 \leq d < 2^K. \quad (\text{B.18})$$

Then $A[d'] \leq A[d]$ whenever $d \subseteq d'$ bitwise. In particular $A[2^k j] \geq A[2^k j + i]$ for all $0 \leq k \leq K$, $0 \leq j < 2^{K-k}$, $0 \leq i < 2^k$, and $A[0] \geq A[d]$ for every d .

Proof. Let $d \subseteq d'$. By Lemma 5, for every pair $u \leq v$ we have $0 \leq \Delta_{uv}(d) \leq \Delta_{uv}(d')$, whence $\phi(\Delta_{uv}(d)) \geq \phi(\Delta_{uv}(d'))$ because ϕ is even and non-increasing on non-negative arguments. Multiply by $w_u w_v \geq 0$ and sum over pairs (the sum over $u > v$ duplicates the sum over $u < v$, since ϕ is even and $\Delta_{vu} = -\Delta_{uv}$). The

dyadic-block statement is the case $d = 2^k j$, $d' = 2^k j + i$, whose bits are disjoint. $\square$

Corollary 8. *The bitwise monotonicity (B.1), and hence the dyadic-block property (B.2), holds for every column of the variance map $A[d_m, \text{subband}, p, F]$ of §D.5, in the no-detrender setting of §B.1, for every `DedispersionTree` (any γ , any e), every subband (any `pf.level` l , both the aligned Case 1 and the half-aligned Case 2), every profile p , and every input channel F .*

Proof. Lemma 1 writes the column in the form (B.18) with $w_u \geq 0$; Lemma 6 gives $\phi = A_p$ the required monotonicity; apply Theorem 7. For $\gamma > 0$ the exported array is $A[d_m] = \tilde{A}[d_m + 2^{K-1}]$, where $\tilde{A}$ satisfies the theorem; since $d_m \subseteq d'_m$ (with both $< 2^{K-1}$) implies $d_m + 2^{K-1} \subseteq d'_m + 2^{K-1}$, the property is inherited. $\square$

Three extensions are immediate, because bitwise monotonicity is preserved by non-negative combinations and by pointwise maxima over a common DM index:

- **Summing over frequency channels.** $y[d_m] = \sum_F V_F A[d_m, \cdot, \cdot, F]$ has the property for any $V_F \geq 0$; this is the vector Av of §D.13.
- **Maximizing over profiles or multiplets.** Arrays which share the same d_m index (e.g. the P profiles) inherit the property under sums and pointwise maxima, since the maximizing index is common to all of them. The multiplets of one subband instead share the *coarse* DM index d_{hi} , and the property passes to them too, because d_{hi} and d_{lo} occupy disjoint bit ranges of $d_m = 2^l d_{hi} + d_{lo}$: each individual multiplet’s array $d_{hi} \mapsto A[2^l d_{hi} + d_{lo}]$ is bitwise monotone, since $d_{hi} \subseteq d'_{hi}$ implies $2^l d_{hi} + d_{lo} \subseteq 2^l d'_{hi} + d_{lo}$; and the array the peak-finder actually forms, $B[d_{hi}] = \max_{d_{lo}} A[2^l d_{hi} + d_{lo}]$, equals $A[2^l d_{hi}]$ by Theorem 7 (as $0 \subseteq d_{lo}$), i.e. it is the fine array sampled at multiples of 2^l , and is therefore bitwise monotone in d_{hi} . (Different subbands have different l and hence different DM ranges, so a maximum over subbands only makes sense once they are put on a common DM grid; on such a grid it inherits the property as well.)
- **Coarse-graining the DM axis.** If the DM axis is coarse-grained by a factor 2^c using `max` (as in the dedispersion output arrays, *dedispersion.pdf*, “Dedispersion output arrays”), the coarse-grained array equals the fine array sampled at $d_m \equiv 0 \pmod{2^c}$, and $i \subseteq i'$ implies $2^c i \subseteq 2^c i'$, so the coarse array inherits the property.

B.5 Sharpness

Neither hypothesis of Theorem 7 can be dropped.

- *Non-negative weights.* Take $K = 1$, $h = [1]$ (profile $p = 0$) and $w = (1, -1)$. Then $A[0] = 0$ and $A[1] = 2$, so even the two-element instance $0 \subseteq 1$ of (B.1) fails. This is why the argument does not survive a `GpuDetrenderLps2d` in front of the dedisperser: the effective per-channel response is no longer a non-negative weight times a single delta, so neither hypothesis of Theorem 7 survives. This is not a gap in the proof but a real failure: computing the detrended variance map by brute force (§C) at rank 6 with `subband_counts=[2,2,1]`, about half of all aligned dyadic blocks of $A[\cdot, \text{subband}, p, F]$ are violated, with block maxima up to 5 times the block-bottom value for an $(n, W) = (2, 4)$ detrender. For the $W = 0$ detrender the failure is extreme and systematic: the detrender annihilates the zero-DM mode, so $A[0]$ is close to zero and is the block *minimum* rather than its maximum.
- *Unimodal kernel autocorrelation.* Take $K = 3$, $w = (1, 1, 1, 1, 1, 1, 1, 1)$ (a full-band, uniform gridding footprint) and the two-hump kernel $h = [1, 0, 0, 0, 1]$, whose autocorrelation $A_h = [2, 0, 0, 0, 1]$ is not monotone. Then $A = (128, 64, 48, 32, 30, 32, 26, 24)$, which violates (B.1) at $4 \subseteq 5$ ($A[5] = 32 > A[4] = 30$), and violates the dyadic-block property on the block $\{4, 5\}$. All four current profile shapes are co-centered sums of boxcars, so Lemma 6 applies; but a future profile which is not (for example a matched filter for a scattered or multi-component pulse) would invalidate the conclusion, not merely the proof – and, as this example shows, for perfectly ordinary gridding weights.

Nor can the bitwise-subset order be enlarged: it is exactly the set of pairs which the hypotheses of Theorem 7 constrain.

Proposition 9 (the order is exactly right). *Fix K and $0 \leq d, d' < 2^K$. The following are equivalent:*

- (i) $A[d'] \leq A[d]$ for every $w \geq 0$ and every even, non-increasing ϕ in (B.18);
- (ii) $\Delta_{uv}(d') \geq \Delta_{uv}(d)$ for every $u \leq v$;
- (iii) $d \subseteq d'$.

Proof. (iii) $\Rightarrow$ (ii) is Lemma 5, and (ii) $\Rightarrow$ (i) is the proof of Theorem 7.

(i) $\Rightarrow$ (ii): the case $u = v$ is trivial, so let $u < v$, and take the two-channel weight vector with $w_u = w_v = 1$ and all other entries zero, so that $A[x] = 2\phi(0) + 2\phi(\Delta_{uv}(x))$; take $\phi(\delta) = 1$ for $|\delta| \leq t$ and $\phi(\delta) = 0$ otherwise, which is even and non-increasing on $\delta \geq 0$. If $\Delta_{uv}(d') < \Delta_{uv}(d)$, then $t = \Delta_{uv}(d')$ gives $A[d'] > A[d]$.

(ii) $\Rightarrow$ (iii): write $\epsilon_m = c^{(m)}(d') - c^{(m)}(d)$ and $D_m = \lfloor d'/2^m \rfloor - \lfloor d/2^m \rfloor$, so that $\epsilon_m = D_m - D_{m+1}$ and $D_K = 0$ by (B.9). By (B.8), $\Delta_{uv}(x) = \sum_m \alpha_m c^{(m)}(x)$ with $\alpha_m = v_{K-1-m} - u_{K-1-m}$; for a pair $u \leq v$ whose highest differing bit is $i_0 = K - 1 - m_0$ we have $\alpha_m = 0$ for $m < m_0$, $\alpha_{m_0} = 1$, and $\alpha_m \in \{-1, 0, 1\}$ arbitrary and independent for $m > m_0$ (the lower bits of u and v are unconstrained). Hence (ii) says exactly

$$\epsilon_{m_0} \geq \sum_{m > m_0} |\epsilon_m| \quad \text{for every } 0 \leq m_0 < K. \quad (\text{B.19})$$

Since the right side is ≥ 0 , every $\epsilon_m \geq 0$, so the constraint becomes $\epsilon_{m_0} \geq \sum_{m > m_0} \epsilon_m = D_{m_0+1}$, i.e. $D_{m_0} \geq 2D_{m_0+1}$. But $D_m - 2D_{m+1} = d'_m - d_m$, so this says $d'_m \geq d_m$ for every m , which is $d \subseteq d'$. $\square$

In particular A need not be monotone in d_m as an integer, and no weaker hypothesis on the DM indices will do. A minimal example within the hypotheses of Theorem 7, using a real peak-finding kernel: $K = 2$, $w = (0, 1, 1, 0)$, $h = B_2 = [1, 1]$ (so $\phi = A_h = [2, 1, 0, \dots]$) gives $A = (8, 6, 8, 6)$, which increases from $d = 1$ to $d = 2$. The reason is visible in (B.8): channels $u = 1$ and $v = 2$ have lags $(1, 0)$ at $d = 1$ but $(1, 1)$ at $d = 2$, so the pair is *less* spread out at the larger DM index – while at $d = 3 = 1|2$, which is above both in the subset order, the separation is back to 1. The same effect is easy to see in the real variance map: for instance in `configs/dedispersion/toy.yml`, tree 0, subband $n = 0$ ($l = 1$, $\rho = 9$), channel $F = 328$, profile $p = 0$, one finds $A[255] = 5.08 \times 10^{-4} < A[256] = 6.71 \times 10^{-4}$, with $A[255 \& 256] = A[0] = 9.77 \times 10^{-4}$ above both.

Finally, the statement is about the *natural* (non-bit-reversed) DM index $d_m = 2^l d_{\text{hi}} + d_{\text{lo}}$, which is the convention of §D.5. Under a bit-reversed DM index, aligned dyadic blocks become strided sets, and (B.1) becomes bitwise monotonicity with respect to the reversed bit labelling (the subset order is preserved by any relabelling of the bits, so the theorem is convention-independent; only its “dyadic block” phrasing is not).

C Brute-force computation of the variance map

Although Eq. (1.6) defines A through a random ensemble, A is really just a statement about the matrix elements of L , and can be computed one column at a time by applying L to “one-hot” input arrays. This appendix works out that brute-force algorithm.

It is very slow, but it treats every stage of L as a black box, so it needs no new analysis to handle subbands, time-downsampled trees, early triggers, or – most importantly – the `GpuDetrenderLps2d`, which the machinery of §D cannot currently handle at all. In fact the relationship is closer than that: §D *is* this algorithm, executed sparsely (propagating the one-hot impulse response through the dedispersion steps in compressed form) instead of densely. The brute-force version is the dense reference that the sparse version should be tested against.

Besides the parameters of §1.1, we will need the `GpuDetrenderLps2d` time half-width W (*detrending.pdf*, “2-d detrending”) and the max peak-finding kernel width $W_{\max}$ (*dedispersion.pdf*, “Peak-finding”).

C.1 A as a set of column norms

Write the matrix elements of L as $L_{\alpha t, Ft'}$, i.e.

$$\text{out}_{\alpha}[t] = \sum_{F=0}^{N_{\text{freq}}-1} \sum_{t'} L_{\alpha t, Ft'} \text{in}[F, t'] \quad (\text{C.1})$$

where $0 \leq \alpha < 2^{r-R}MP$ is the flattened output index, t is a downsampled output time, and t' is an undownsamped input time. Since the input has covariance $\langle \text{in}[F, t'] \text{in}[F', t''] \rangle = v_F \delta_{FF'} \delta_{t't''}$,

$$y_{\alpha} = \langle \text{out}_{\alpha}[t]^2 \rangle = \sum_F v_F \sum_{t'} L_{\alpha t, Ft'}^2 \quad (\text{C.2})$$

and comparing with Eq. (1.6),

$$A_{\alpha F} = \sum_{t'} L_{\alpha t, Ft'}^2 \quad (\text{C.3})$$

for any output time t which has reached steady state. That is, **a column of A is a set of squared column norms of L** , taken over the input time axis at fixed output time.

Now let $e^{(F, t_0)}$ denote the one-hot input array $\text{in}[F', t'] = \delta_{FF'} \delta_{t't_0}$. Applying L to it picks out a *row* of L , not a column:

$$(L e^{(F, t_0)})_{\alpha}[t] = L_{\alpha t, Ft_0} \quad \implies \quad \sum_t (L e^{(F, t_0)})_{\alpha}[t]^2 = \sum_t L_{\alpha t, Ft_0}^2 \quad (\text{C.4})$$

So the proposed algorithm computes a *row* norm where Eq. (C.3) asks for a *column* norm. The two agree by time-translation invariance – but the details differ between $\gamma = 0$ and $\gamma > 0$, which is the one place where the algorithm needs care.

C.2 Undownsamped trees ($\gamma = 0$)

Every stage of L is time-translation invariant. The `GpuDetrenderLps2d` run with no mask is a fixed FIR filter along time: with no mask its normal equations, and hence its mask expansion and its conditioning cuts, depend on neither the data nor t , so L really is both linear and shift-invariant. Tree gridding acts only on the frequency axis, and tree dedispersion and the peak-finding convolutions are sums of lagged copies. Therefore

$$L_{\alpha t, Ft'} = \ell_{\alpha F}[t - t'] \quad (\text{C.5})$$

and Eqs. (C.3), (C.4) both collapse to $\sum_u \ell_{\alpha F}[u]^2$. Hence, for any t_0 satisfying the support conditions of §C.4 below,

$$A_{\alpha F} = \sum_t (L e^{(F, t_0)})_{\alpha} [t]^2 \quad (\text{C.6})$$

i.e. apply L to a one-hot input, square, and sum over the output time axis. One pass per input channel gives all N_{freq} columns of A .

C.3 Downsampled trees ($\gamma > 0$)

For $\gamma > 0$, the operator L contains a time-downsampling stage $\text{TDS}(\gamma)$ (*dedispersion.pdf*, “Downsampled trees and the LaggedDownsampler”) between the detrender and the tree, and is therefore invariant only under input shifts by 2^γ samples. The right generalization is a *polyphase* decomposition of the input time index into a downsampled index and a phase,

$$t' = 2^\gamma s + c, \quad 0 \leq c < 2^\gamma \quad (\text{C.7})$$

Invariance under $s \rightarrow s + 1$ (which shifts the output by one downsampled sample) gives

$$L_{\alpha t, F(2^\gamma s + c)} = \ell_{\alpha F}^{(c)}[t - s] \quad (\text{C.8})$$

with 2^γ generally-distinct impulse responses $\ell^{(0)}, \dots, \ell^{(2^\gamma-1)}$. Eq. (C.3) now reads

$$A_{\alpha F} = \sum_{c=0}^{2^\gamma-1} \sum_s \ell_{\alpha F}^{(c)}[t - s]^2 = \sum_{c=0}^{2^\gamma-1} \|\ell_{\alpha F}^{(c)}\|^2 \quad (\text{C.9})$$

whereas a single one-hot at input time t_0 sees only the phase $c = t_0 \bmod 2^\gamma$. So a downsampled tree does need 2^γ one-hot arrays per input channel, one per residue class of t_0 modulo 2^γ , and the results are *summed* (not averaged):

$$A_{\alpha F} = \sum_{c=0}^{2^\gamma-1} \sum_t (L e^{(F, t_c)})_{\alpha} [t]^2, \quad t_c \equiv c \pmod{2^\gamma} \quad (\text{C.10})$$

Any representative t_c of each residue class will do; e.g. $t_c = t_0 + c$ with t_0 a multiple of 2^γ .

There is one important special case. If the part of L upstream of $\text{TDS}(\gamma)$ is *instantaneous* in time – no detrender at all, or a `GpuDetrenderLps2d` with $W = 0$, which fits each time sample independently and is therefore a pure frequency-axis operator – then $\text{TDS}(\gamma)$ maps $e^{(F, 2^\gamma s + c)}$ to the *same* downsampled array for every c , the 2^γ phases coincide, and one pass suffices:

$$A_{\alpha F} = 2^\gamma \sum_t (L e^{(F, t_0)})_{\alpha} [t]^2 \quad (W = 0) \quad (\text{C.11})$$

This is the familiar statement that downsampling with a $2^{-\gamma/2}$ normalization leaves the variance unchanged (*dedispersion.pdf*, “Time-downsampled trees and early triggers”), and it is worth keeping as a unit test: with $W = 0$ the 2^γ phase outputs must agree *exactly*. For $W > 0$ they genuinely differ, since the polyphase components of the detrender’s time response differ, and all 2^γ passes are needed.

C.4 Placing the one-hot, and sizing N_{time}

Eqs. (C.6), (C.10) are exact only if the *entire* impulse response is emitted: anything clipped at either end of the array is silently dropped from the sum of squares, and the result is an underestimate with no error indication. Three conditions.

- **Detrender padding.** The `GpuDetrenderLps2d` is handed a length- $(T + 2W)$ buffer and overwrites only samples $[W, W + T)$ with the residual (*detrending.pdf*, “2-d detrending”). A one-hot at buffer index t_0 produces a detrended response on $[t_0 - W, t_0 + W]$, all of which must lie in the emitted region, so we need $t_0 \geq 2W$.
- **No negative times.** Tree dedispersion defines $\text{in}[\cdots, t] = 0$ for $t < 0$ (*dedispersion.pdf*, “Tree dedispersion”), so a response reaching back before the start of the stream would be truncated – equivalently, we would be computing a non-steady-state variance (cf. the footnote to Eq. D.3). The previous condition already rules this out, since the response starts at $t_0 - W \geq W \geq 0$.
- **Enough room downstream.** Going forward from the one-hot, the response is spread by three stages:
 - the detrender, over $2W + 1$ undownsamped samples, hence $\lfloor 2W/2^\gamma \rfloor + 1$ downsampled samples after $\text{TDS}(\gamma)$;
 - (subbanded) tree dedispersion, by at most Δ_{dd} downsampled samples, where

$$\Delta_{\text{dd}} \equiv \begin{cases} 2^r & \text{if } \gamma = 0 \\ 2^{r+1} & \text{if } \gamma > 0 \end{cases} \quad (\text{C.12})$$

- the peak-finding convolution, by at most $T_{\text{max}} - 1 = 2W_{\text{max}} - 1$ downsampled samples (§D.2). If the kernel applies a per-profile time shift (*dedispersion.pdf*, “Dedispersion output arrays”), that only translates the response, which is harmless here.

Eq. (C.12) deserves a comment. Δ_{dd} is simply the largest full-band delay searched by the tree, because the output time index is by construction the arrival time extrapolated to the top of the tree’s own band (*dedispersion.pdf*, “Subband search”) – so the per-subband time lags T do not *add* to this bound, they are part of it. Concretely, in Case 1 of subbanded dedispersion (*dedispersion.pdf*, “Subband search”) the total lag from tree-freq f to the output is at most

$$\underbrace{T}_{= d_{\text{hi}} 2^l (2^{R-l} - 1 - f)} + \underbrace{d}_{= d_{\text{hi}} 2^l + d_{\text{lo}}} \leq d_{\text{hi}} 2^R + d_{\text{lo}} \leq (2^{r-R} - 1) 2^R + (2^l - 1) < 2^r \quad (\text{C.13})$$

(maximized at $f = 0$, and using the fact that the largest within-subband lag is $L_{r-R+l}(0, d) = d$), and Case 2 is the same. The two cases in Eq. (C.12) differ because for $\gamma > 0$ the tree keeps only the upper DM half (*dedispersion.pdf*, “Time-downsampled trees and early triggers”), so the tree rank r is one less than the rank of the underlying DD: in the code, $r = \text{total_rank} = r_{\text{top}} - e - [\gamma > 0]$, and $\Delta_{\text{dd}} = 2^{r_{\text{top}} - e}$ downsampled samples in both cases.

Adding the three contributions, the response occupies at most

$$n_t = \left\lfloor \frac{2W}{2^\gamma} \right\rfloor + \Delta_{\text{dd}} + 2W_{\text{max}} \quad (\text{C.14})$$

downsampled output samples starting from the one-hot, so it suffices to take

$$2W \leq t_0 < 2W + 2^\gamma \quad N_{\text{time}} \geq 4W + 2^\gamma (\Delta_{\text{dd}} + 2W_{\text{max}} + 1) \quad (\text{C.15})$$

where the $4W$ is $2W$ from the placement of t_0 , plus W for the detrender’s forward reach, plus W of trailing detrender padding; and the “+1” inside the parentheses absorbs the choice of phase c .

Two practical remarks. First, because the answer is a sum over *all* output times, the algorithm is insensitive to how the stream is chunked: one may equally run the dedisperser over several chunks totalling N_{time} samples and accumulate $\sum_t (\cdots)^2$ across them, and there is no need to know when each DM row reaches steady state. Second, the cheapest guard against a mis-sized array is to assert that the last few emitted output samples are identically zero; Eq. (C.15) is a bound, not a tight estimate, and it is easy to get wrong by a factor of 2^γ .

C.5 Cost, and relation to the other algorithms

Each pass costs one full evaluation of L on an $(N_{\text{freq}}, N_{\text{time}})$ array, and $N_{\text{freq}}2^\gamma$ passes are needed per tree (N_{freq} passes if $W = 0$). For a CHORD-sized config that is 28160 passes per tree, each dedispersing $\sim 10^5$ downsampled time samples at rank $r = 16$: hopelessly slow. Brute force is a small-config reference implementation, not a production algorithm, and not a practical substitute for §D, which runs at full CHIME/CHORD scale. Two things make it less bad than it looks: passes for different F can be batched along the dedisperser’s spectator beam axis, and almost all of the work in each pass is on identically-zero data, which a specialized implementation could skip. Skipping it systematically is precisely what §D.4 does.

Three remarks on how this fits with the rest of the notes.

- **It generalizes the Monte Carlo check** of §D.14. That check feeds random Gaussian noise with per-channel variance V_F through a `ReferenceDedisperser` and compares empirical variances against the analytic prediction. The brute-force algorithm runs the same forward machinery, but replaces the random ensemble by the deterministic standard basis $\{e^{(F, t_0)}\}$ – so the result is exact rather than noisy, and it yields the full matrix A rather than a single product Av .
- **It should agree with §D** wherever both apply. With the detrender disabled, Eq. (C.6) must reproduce Eq. (D.3) column by column: indeed $A_{(d,p),F} = \|h_p * G_{Fd}\|^2$ is exactly the statement that the two agree. This is the strongest end-to-end test we have, since the two computations share no code, and §D.14 describes how it is run.
- **It is the only algorithm we have which handles the detrender.** The efficiency of §D.4 rests on the gridding footprint $\mathcal{S}_F$ being a contiguous range of a few tree-freq channels. A `GpuDetrenderLps2d` destroys this: the detrended response to a one-hot in channel F is spread over every channel of F ’s spline zone (*detrending.pdf*, “2-d detrending”), so the `SparseTile` representation would degrade towards a dense one. Brute force is completely insensitive to it. Note also that these off-diagonal responses are individually small but not negligible in aggregate: each contributes $O(\lambda_F^2)$ to A , but there are $O(N_{\text{freq}})$ of them, so they are a percent-level effect rather than a rounding error.

Finally, three checks worth asserting in any implementation. Every entry of A is a sum of squares, so $A_{\alpha F} \geq 0$ identically. For an early-triggered tree ($e > 0$, and likewise for a tree whose subbands do not span the full band), the columns of all input channels outside the tree’s frequency range must vanish identically. And the arithmetic should be done in float64: the one-hot response has amplitude $O(2^{-r/2})$ before squaring, and the detrender’s off-diagonal contributions are smaller still, so in float32 the aggregate effect noted above would be at risk of being lost to roundoff against the diagonal term.

D Detrender-free computation of the variance map

This appendix documents `pirate_frb/varmap/detrender_free.py`, which computes the variance map A of Eq. (1.6) analytically – no dedisperser is run and no GPU is used – and returns it directly in low-rank form $A = QW^T$. (It does build a `DedispersionPlan`, but a `Params::minimal()` one, which is pure arithmetic: no ring buffer, no kernel params, no CUDA device. That is the only way to get at a config’s `DedispersionTrees`.) It is the algorithm behind `pirate_frb varmap df`.

The no-detrender hypothesis is load-bearing, not a missing feature. Throughout this appendix the linear operator L of §1.1 is taken with the `GpuDetrenderLps2d` step omitted, so “ A ” here means the A of Eq. (1.6) for bare tree dedispersion. Three separate steps below are false with a detrender in front, and each says so where it is used: the sparse propagation of §D.4 (whose efficiency needs a contiguous, narrow gridding footprint), the coarse-graining slice of §D.10, and the row restriction of §D.12. The brute-force algorithm of §C is the one that handles a detrender, at enormously greater cost.

There are three entry points, and they differ in what they materialize:

- `compute_detrender_free_base_map(config)` returns the map A of the base tree, as a `VarianceMap` in factored form. This is the most expensive of the three, because it is the only one that materializes Q .
- `compute_detrender_free_multi_map(config)` returns a `VarianceMultiMap` covering every tree of the config, by computing the base map once and slicing it (§D.12).
- `compute_detrender_free_varfine(config, v)` and its coarse-grained sibling `compute_detrender_free_varcoarse` return the *vector* Av for given input-channel variances v_F , for every tree, without ever forming A (§D.13). This is what the real-time server consumes, and it has a C++ port (`src_lib/varmap.cpp`) which is what actually runs in production.

The three share one implementation, the class `SdPlan`, whose four passes are §D.7.

D.1 Statement of the problem

The dedisperser input is a shape $(N_{\text{freq}}, N_{\text{time}})$ array `in[F, t]`. We assume that its elements are uncorrelated, with a variance which depends on frequency but not time:

$$\langle \text{in}[F, t] \text{in}[F', t'] \rangle = V_F \delta_{FF'} \delta_{tt'} \quad (\text{D.1})$$

The three stages downstream – tree gridding (*dedispersion.pdf*, “Tree gridding kernel”), tree dedispersion (*dedispersion.pdf*, “Tree dedispersion”), and convolution with the peak-finding kernels h_p (*dedispersion.pdf*, “Peak-finding”) – are all linear, and are all time-translation invariant, so their composition can be written as

$$y_p[d, t] = \sum_{F=0}^{N_{\text{freq}}-1} \sum_{t'} (h_p * G_{Fd})[t - t'] \text{in}[F, t'] \quad (\text{D.2})$$

Here, G_{Fd} is the *impulse response*: the tree dedispersion output at delay d , if gridding and $\text{DD}(r)$ are run on an input array whose only nonzero element is `in[F, 0] = 1`. Combining with Eq. (D.1), the array we want to compute is

$$\text{Var}[d, p] \equiv \langle y_p[d, t]^2 \rangle = \sum_F V_F \|h_p * G_{Fd}\|^2 \quad \text{where } \|g\|^2 \equiv \sum_t g[t]^2 \quad (\text{D.3})$$

Note that the RHS does not depend on t , i.e. every output time sample has the same variance.³ Comparing with Eq. (1.6), the matrix element of the variance map is

$$A_{\alpha F} = \|h_p * G_{Fd}\|^2 \quad (\text{D.4})$$

so computing A is exactly computing $\|h_p * G_{Fd}\|^2$ for every (F, d, p) .

We can make Eq. (D.3) completely explicit. Recall from *dedispersion.pdf*, “Non-recursive formulation of tree dedispersion” that $\text{DD}(r)$ maps a one-hot tree-freq channel f to a single sample, at lag $L_r(f, d)$ with amplitude $2^{-r/2}$, and from *dedispersion.pdf*, “Tree gridding kernel” that input channel F is gridded onto tree-freq channels with weights w_{fF} . Therefore

$$G_{Fd}[t] = 2^{-r/2} \sum_{f \in \mathcal{S}_F} w_{fF} \delta_{t, L_r(f, d)} \quad (\text{D.5})$$

where the *gridding footprint* $\mathcal{S}_F \equiv \{f : w_{fF} > 0\}$ is the set of tree-freq channels whose interval I_f overlaps the input channel $[F, F + 1)$. Since the intervals I_f tile the band, $\mathcal{S}_F$ is a contiguous range

$$f_0(F) \leq f < f_1(F) \quad (\text{D.6})$$

which is usually just a few channels wide (in the CHORD config, $N_{\text{freq}} = 28160$ input channels are gridded onto $2^r = 65536$ tree-freq channels).

Thus the impulse response of a single input channel is a handful of delta functions, and Eqs. (D.3), (D.5) are a complete specification of the answer. However, evaluating them directly, for each (F, d) pair, would be too slow ($N_{\text{freq}} 2^r \approx 2 \times 10^9$ pairs in the CHORD config), and would also be wasteful: we will see that the 2^r delays give rise to far fewer than 2^r distinct answers.

D.2 Variance from autocorrelations

The lowest-level building block is the map $g \mapsto \|h_p * g\|^2$, for a short time series g and each of the P peak-finding kernels. Expanding the convolutions and collecting terms by lag,

$$\|h_p * g\|^2 = \sum_{\delta} \rho_g[\delta] A_p[\delta] = \rho_g[0] A_p[0] + 2 \sum_{\delta \geq 1} \rho_g[\delta] A_p[\delta] \quad (\text{D.7})$$

where ρ_g and A_p are the (one-sided) autocorrelations of the data and of the kernel:

$$\rho_g[\delta] = \sum_t g[t] g[t + \delta] \quad A_p[\delta] = \sum_t h_p[t] h_p[t + \delta] \quad (\text{D.8})$$

Two features of Eq. (D.7) are worth noting.

- $A_p[\delta]$ vanishes for $\delta \geq \text{len}(h_p)$, so lags $\delta \geq T_{\text{max}} \equiv \max_p \text{len}(h_p) = 2W_{\text{max}}$ never contribute. The class `PfVarianceConvolver` precomputes the shape (P, T_{max}) table $A_p[\delta]$ once, and Eq. (D.7) then becomes a small matrix multiply which returns all P profiles at once. The $\delta = 0$ column is just the kernel norms:

$$A_p[0] = \|h_p\|^2 = 2^\lambda \times \{1, 2, \frac{3}{2}, \frac{5}{2}\}_q \quad (\text{D.9})$$

where $p = 3\lambda + q$ as in *dedispersion.pdf*, “Peak-finding”. Evaluating (D.7) for a time series whose support has length n_t costs $O(n_t \min(n_t, T_{\text{max}}))$ for ρ_g , plus $O(\min(n_t, T_{\text{max}}) P)$ for the contraction.

³This is a steady-state statement. Near the beginning of a run, output samples have a *smaller* variance, since part of the impulse response reaches back to times before the start of the stream (recall from *dedispersion.pdf*, “Tree dedispersion” that `in[ $\dots$ ,  $t$ ]` is defined to be zero for $t < 0$). The `DedispersionPlan` knows when each DM row reaches steady state (`compute_steady_state_it0`), and the Monte Carlo check of §D.14 uses this.

- The LHS is invariant under a time translation $g[t] \rightarrow g[t - T]$. Therefore, we only need each impulse response *up to an arbitrary time shift*. This is the key to the compression in §D.3–D.4: most of the d -dependence of G_{Fd} is a pure time shift, and can simply be discarded.

One consequence is worth stating early, because two later steps depend on it. The kernel autocorrelations $A_p[\delta]$ are non-negative and non-increasing in δ , which holds because every profile is a co-centered non-negative sum of boxcars. That is a hypothesis of the monotonicity theorem of §B, which §D.10 uses, so it is not a curiosity: it is checked by a unit test which fails with an explanation if the kernel bank ever stops having the property.

D.3 Which delay bits matter

By the time-translation invariance just noted, only the *relative* lags within the footprint matter in Eq. (D.5). From the definition of the lag function (*dedispersion.pdf*, “Non-recursive formulation of tree dedispersion”), the relative lag of two tree-freq channels f, f' is

$$L_r(f, d) - L_r(f', d) = \sum_{i=0}^{r-1} \sum_{j=0}^{r-1} \Omega_{i+j-r} (f'_i - f_i) d_j \quad (\text{D.10})$$

Suppose that f and f' agree in all bits above position i_0 , so that only terms with $i \leq i_0$ survive in Eq. (D.10). Since $\Omega_n = 0$ for $n < -1$, a term also vanishes unless $j \geq r - 1 - i$. Therefore the relative lag depends only on the bits d_j with $j \geq r - 1 - i_0$, i.e. on the top $(i_0 + 1)$ bits of d . For a contiguous footprint, all channels agree in the bits above

$$i_0(F) \equiv \left\lfloor \log_2 (f_0(F) \oplus (f_1(F) - 1)) \right\rfloor \quad (\text{D.11})$$

where $\oplus$ denotes bitwise XOR (and we define $i_0 = -1$ if the footprint is a single channel). We conclude:

The contribution of input channel F to Eq. (D.3) depends on the delay d only through its top $(i_0(F) + 1)$ bits.

In particular, if the footprint is a single tree-freq channel f , then the contribution is d -independent, and Eqs. (D.3), (D.5) collapse to

$$V_F \|h_p * G_{Fd}\|^2 = 2^{-r} V_F w_{fF}^2 \|h_p\|^2 \quad (\text{D.12})$$

Eq. (D.11) has a simple interpretation: $i_0(F)$ is the largest i such that the footprint straddles a boundary between two “coarse” frequency channels of width 2^i (cf. the coarse-freq blocking of *dedispersion.pdf*, “Tree dedispersion”). Equivalently, $\text{DDS}(i_0)$ is the last dedispersion step which combines two nonzero halves of the footprint with each other. There are only 2^{r-i} such boundaries in the band, and each one is straddled by at most one input channel, so at most $O(2^{r-i})$ input channels have $i_0 \geq i$: large $i_0(F)$ is rare, but expensive.

This bit set is central to the algorithm, and the code carries it as a bitmask called `dbits`, with `|dbits|` its popcount. Two properties are used repeatedly. First, it can be computed in closed form from (f_0, f_1) alone, with no iteration and no data (`SparseTile._predict_dbits()`); the planning pass of §D.7 runs on that closed form. Second, the closed form is an upper bound which the actual propagation of §D.4 can improve on, since relative lags sometimes cancel. The two are reconciled by an assertion at the point where the two meet, which is what makes the plan and the data agree by construction rather than by hope.

D.4 Sparse propagation of the impulse response

Recall from *dedispersion.pdf*, “Tree dedispersion” that the intermediate array after k dedispersion steps has shape $(2^{r-k}, 2^k, N_{\text{time}})$, with axes (coarse-freq, delay, time). For a one-hot input, such an array is highly structured, and the class `SparseTile` represents it with the following members:

- f_0, n_f : the array vanishes outside coarse-freq channels $f_0 \leq f < f_0 + n_f$.

- n_t : the stored data vanishes outside time samples $0 \leq t < n_t$.
- **dbits**: a bitmask selecting a subset of the k delay bits (the “content bits”).
- **data**: an array of shape $(n_f, 2^{|\text{dbits}|}, n_t)$.
- t_0 and $\tau_0, \dots, \tau_{k-1}$: a constant time shift, and one time shift per delay bit.
- s : a scalar multiplier.

The dense array represented by the tile is

$$\text{arr}[f, d, t] = s \cdot \text{data}[f - f_0, d|_{\text{dbits}}, t - T(d)] \quad T(d) = t_0 + \sum_{i=0}^{k-1} d_i \tau_i \quad (\text{D.13})$$

where $d|_{\text{dbits}}$ denotes the selected bits of d , packed (highest bit first) into an index $0 \leq d|_{\text{dbits}} < 2^{|\text{dbits}|}$. The compression is therefore two-fold: the array is stored only on its (n_f, n_t) support, and its 2^k delay rows are represented by $2^{|\text{dbits}|}$ stored rows, plus a time shift $T(d)$ which is affine in the bits of d .

The starting point ($k = 0$) is the gridding output for a one-hot input, which is a tile with $f_0 = f_0(F)$, $n_f = f_1(F) - f_0(F)$, $n_t = 1$, **dbits** = 0, $s = 1$, no shifts, and $\text{data}[f, 0, 0] = w_{fF}$.

The tile representation is preserved by a dedispersion step $\text{DDS}(k)$. Consider an output coarse-freq channel f , whose inputs are the “lower” channel $2f$ and the “upper” channel $2f + 1$, and write the output delay index as $d' = 2d + b$ (so that $d'_0 = b$ and $d'_{j+1} = d_j$). There are three cases:

- **Upper half only** (the lower half is outside the footprint). Then $\text{out}[f, 2d + b, t] = 2^{-1/2} \text{in}[2f + 1, d, t]$ for both $b = 0, 1$, i.e. the output does not depend on the new delay bit at all. The tile is unchanged, except that its bitmask and shift vector are “lifted” by one bit,

$$\text{dbits} \rightarrow 2\text{dbits} \quad (\tau_0, \dots, \tau_{k-1}) \rightarrow (0, \tau_0, \dots, \tau_{k-1}) \quad (\text{D.14})$$

and $s \rightarrow s/\sqrt{2}$. No data is touched.

- **Lower half only**. Then $\text{out}[f, 2d + b, t] = 2^{-1/2} \text{in}[2f, d, t - d - b]$, i.e. the output depends on the new delay bit only through a time shift. The bitmask and shift vector are lifted as in Eq. (D.14), and in addition the dedispersion lag $(d + b)$ is absorbed into the shifts, by adding

$$(1, 1, 2, 4, \dots, 2^{k-1}) \quad (\text{D.15})$$

to the (lifted) length- $(k + 1)$ shift vector, using $d + b = b + \sum_{j \geq 1} d'_j 2^{j-1}$. Again no data is touched.

- **Both halves**. Now the output is a sum of two “legs”, each of which is a lifted copy of one input tile, and each of which carries its own shift vector: σ^U for the upper leg, and σ^L for the lower leg (which includes the extra dedispersion lags of Eq. D.15). Only the *difference* between the two shift vectors is physical, so we factor the common part $\tau_j = \min(\sigma_j^L, \sigma_j^U)$ out into the output tile’s shifts, and fold the residuals $(\sigma_j^L - \tau_j)$ and $(\sigma_j^U - \tau_j)$ into the data, by materializing each output row and adding the two legs at their residual offsets (the constant shifts t_0 are treated the same way). A delay bit j becomes a content bit of the output if the two legs’ residuals differ ($\sigma_j^L \neq \sigma_j^U$), or if it was already a content bit of either leg.

This is the only case in which content bits are created, data is copied, and the time support grows (by up to $\sum_j (\sigma_j^L - \tau_j) \leq 2^k$).

Since the footprint is contiguous, at most two coarse-freq channels – the first and the last – can have a missing half. The class `SparseTileTriple` exploits this by splitting the array into (at most) three tiles: first channel, bulk, last channel. The two edge tiles can then take the cheap single-leg cases, while the bulk is processed with the both-halves case uniformly.

After enough dedispersion steps only one coarse-freq channel is left, and the resulting tile ($n_f = 1$) is an exact, compressed representation of the impulse response. Its content bits are always a subset of the bits predicted in §D.3, and are sometimes a proper subset, since the merge rule above only marks a bit when the two legs’ residual shifts actually differ.

Finally, since $\|h_p * g\|^2$ is invariant under time translation, the shifts $T(d)$ in Eq. (D.13) drop out of the variance entirely, leaving

$$\|h_p * G_{Fd}\|^2 = s^2 \|h_p * \text{data}[0, d|_{\text{dbits}}, \cdot]\|^2 \quad (\text{D.16})$$

So we run each of the $2^{|\text{dbits}|}$ stored rows through Eq. (D.7), and obtain a shape $(2^{|\text{dbits}|}, P)$ array which is the entire contribution of one input channel to Eq. (D.3), for one subband. The factor s^2 is easy to overlook and is not optional: an edge tile defers its $1/\sqrt{2}$ into s rather than touching data, so dropping s^2 is silently wrong exactly where the single-leg cases fire.

D.5 Index conventions, and what a group holds

So far we have suppressed subbands. Reinstating them, the output of a tree is indexed by a multiplet $0 \leq m < M$ and a coarse DM $0 \leq d < 2^{r-R}$, and a multiplet is a (subband, fine DM) pair: subband $0 \leq n < N$ has a level l_n , occupies 2^{l_n} consecutive multiplets starting at $m_{\text{base}}(n)$, and spans 2^{c_n} tree-freq channels with $c_n \equiv r - R + l_n$ (*dedispersion.pdf*, “Subband search”). So

$$m = m_{\text{base}}(n) + e, \quad 0 \leq e < 2^{l_n} \quad (\text{D.17})$$

and the flattened row index of §1.1 is $\alpha = (dM + m)P + p$.

The delay bits of §D.3 refer to a subband’s own dedispersion tree, which has depth c_n , not r . Rows coming from subbands of *different* levels have to be stored in a common labelling, and the one the code uses is **level** r : the level- r delay index of the output element (d, m, p) with $m = m_{\text{base}}(n) + e$ is

$$d_{\text{full}} = (d \ll R) | (e \ll (R - l_n)) \quad (\text{D.18})$$

Lifting a tile from its own level c_n to level r is $r - c_n = R - l_n$ single-leg dedispersion steps, and by the first two cases of §D.4 such a step leaves **data** untouched, shifts **dbits** left by one, and multiplies s by $1/\sqrt{2}$. Hence

$$\text{dbits}(\text{level } r) = \text{dbits}(\text{level } c_n) \ll (R - l_n), \quad \text{var}(\text{level } r) = \text{var}(\text{level } c_n) \cdot 2^{-(R-l_n)} \quad (\text{D.19})$$

Every row is stored with the level- r normalization of the second equation, and the factor 2^{R-l_n} is undone per subband at the very end (§D.8). It cannot be folded into the stored row, precisely because two subbands sharing a row have different l_n . Omitting it is not a subtle error: measured on `toy.yml` it moves the answer by 75%.

D.6 Grouping: entries, the sdbits key, and straddled subbands

The central efficiency idea is that one tile computation and one convolution should serve as many output elements as possible. Fix an input channel F with (unclipped) footprint $[j_0, j_1]$ in tree-freq units. For each subband n , the part of the footprint that subband sees is the intersection

$$[\text{lo}, \text{hi}] = [j_0, j_1] \cap [I_{\text{lo}}(n), I_{\text{hi}}(n)) \quad (\text{D.20})$$

and the subband contributes nothing if this is empty. The observation is that two subbands seeing the *same* range $[\text{lo}, \text{hi}]$ produce the *same* tile and therefore the same variance row, up to the per-subband normalization of Eq. (D.19) which is applied later. So the algorithm pools them: an **entry** is a (channel, set-of-subbands) pair sharing one range, and it produces exactly one tile and one convolution.

Measured, this is 2.9 to 5.7 times fewer convolutions than computing one tile per (channel, multiplet) pair.

Entries are keyed by a single integer, `sdbits`, which packs two fields:

$$\text{sdbits} = (\text{dbits} \ll 42) \mid \text{sbits} \quad (\text{D.21})$$

where `dbits` is the level- r delay-bit mask and `sbits` has bit n set iff subband n is served. The split at 42 is chosen so the key fits a `uint64` in the C++ port: $N \leq 42$ subbands and $r \leq 16$, and both bounds are asserted up front, since exceeding either would corrupt keys silently rather than fail. Entries sharing a key are pooled into one **group**, which is what the SVD of §D.8 acts on.

Half-aligned subbands need their own branch. A subband is *aligned* (Case 1) if $I_{\text{lo}}(n)$ is a multiple of 2^{c_n} , so that the subband is a node of the toplevel dedispersion tree and ordinary aligned iteration reproduces its merges. A subband can instead be *half-aligned* (Case 2, *dedispersion.pdf*, “Subbanded dedispersion”), starting at an odd multiple of 2^{c_n-1} . Then the subband’s own top merge joins the two blocks either side of its midpoint $I_{\text{mid}}(n)$, and those are *not* an aligned pair of the toplevel tree: ordinary iteration would merge each with its absent neighbour instead. When such a subband’s footprint *straddles* its own midpoint ($\text{lo} < I_{\text{mid}}(n) < \text{hi}$), the entry is put on a separate *straddled plan*, iterated only $c_n - 1$ times, and finished with an explicit merge of the two blocks indexed off the midpoint.

Two remarks. First, a straddling subband always gets a row to itself, so the two fields of its `sdbits` are redundant with each other – which the code checks. Second, the branch is rare but not negligible: on `toy.yml` it is 1 row in 645, and omitting it moves the answer by 1.6×10^{-3} . It is also the reason the delay bits of §D.3 are computed in *subband-local* coordinates $f - I_{\text{lo}}(n)$: subtracting $I_{\text{lo}}(n)$ (a multiple of 2^{c_n-1} in every case) preserves the block structure at every level below c_n , so in local coordinates a subband’s dedispersion tree is the ordinary aligned tree, in Case 2 as much as in Case 1.

D.7 The four passes

The class `SdPlan` computes everything except the final lift, in four passes over the input channels. The split exists because the sizes of all the accumulators are known before any data is touched, so nothing has to grow.

1. **Planning.** For each channel and each subband it sees, compute $[\text{lo}, \text{hi})$ and the level- r `dbits` from the closed form of §D.3, classify the entry as straddled or not, and pool subbands sharing a range into one entry. This pass touches no data at all: it is pure integer arithmetic on footprints.
2. **Sizing.** Count how many rows each group will receive, and allocate one dense (capacity, $D \cdot P$) matrix per group, where $D = 2^{|\text{dbits}|}$. Because the planning pass is exact, capacity is a count and not an upper bound.
3. **Tiles.** Walk both plans, build each entry’s tile by the propagation of §D.4, convert it to variances with Eq. (D.7), apply the level- r normalization of Eq. (D.19), and write one row into the group’s matrix. This is where essentially all the arithmetic happens.
4. **Factorization.** SVD each group’s matrix and truncate (§D.8, §D.9).

Two accumulators are filled in parallel, and they do *not* share a key space, which is the one thing to get right when reading the code:

- `sd_matrices`, the per-group dense matrices, keyed by the *coarse* `sdbits` (§D.10) and holding the possibly-sliced, *unweighted* rows;
- `sd_vectors`, keyed by the *full* `sdbits` and holding the row *sums*, weighted by V_F . These are what produce y^{true} , which is defined at fine granularity whatever the map’s coarse-graining rank (§D.10), and they are also the whole of the map-free route of §D.13.

At coarse-graining rank $L = R$ the two key spaces coincide, so indexing one with the other’s key would pass every test that does not set L – and with L set it is a quiet wrong answer rather than a crash, since a coarse key is a syntactically valid full key. Hence the code names which key it is using at every use site, and asserts that the full key space is never smaller than the coarse one.

D.8 Per-group SVD, and the lift to Q and W

Each group holds a dense matrix whose rows are input channels and whose columns are (delay, profile) pairs – that is, the *transpose* of the corresponding block of A . The map is numerically low-rank because the variance varies smoothly with input channel: two neighbouring channels have nearly the same footprint and hence nearly the same impulse response. So we take a truncated SVD in the transposed convention

$$\text{dense_matrix}^T \approx Q_{\text{grp}} W_{\text{grp}}^T \quad (\text{D.22})$$

with Q_{grp} of shape (DP, K) and W_{grp} of shape (F, K) , splitting $\sqrt{\sigma_k}$ between the two factors. (Neither factor is semiorthogonal afterwards, which costs nothing here: the globally stacked factors are not orthogonal anyway, since different groups overlap in their rows of A .) A transpose slip in Eq. (D.22) is silent whenever $DP = F$, since the shapes still match, so it is checked by a test rather than left to inspection.

The groups are then stacked. Each group owns a disjoint block of K columns, so $K_{\text{tot}} = \sum_{\text{groups}} K$, and:

- **W is a scatter.** $W[\text{freq_indices}, k_0:k_0+K] = W_{\text{grp}}$. This is a plain scatter because the input channels within one group are distinct – a property of the plan, which the planning pass checks in its debug mode.
- **Q is a gather, with the per-subband factor.** For each subband n served by the group, and each (d, e) , the level- r index d_{full} of Eq. (D.18) is mapped through the group’s `dbits` to pick the stored row, and the result is multiplied by 2^{R-l_n} to undo Eq. (D.19):

$$Q[(d, m_{\text{base}}(n) + e, p), k_0+k] = 2^{R-l_n} Q_{\text{grp}}[(d_{\text{full}}|_{\text{dbits}}, p), k] \quad (\text{D.23})$$

These writes are assignments rather than accumulations: each group owns a disjoint column block, and within a group the subbands own disjoint multiplet ranges.

The result is $A \approx QW^T$ with Q of shape $(n_\beta, K_{\text{tot}})$ and W of shape $(N_{\text{freq}}, K_{\text{tot}})$, which is returned as a `VarianceMap`.

The lift is where the memory goes. The per-group matrices are small – at `chord_sb2_et.yml` the first three passes run in 14 seconds and hold 17 MB of rows – but Q is dense and $O(n_\beta K_{\text{tot}})$, which is 63.8 GiB for a fine map at that config. §D.10 and §D.13 are the two ways around that, and they are the reason `SdPlan` is a class whose constructor does *not* lift: a caller who wants only the per-group factors, or only Av , would otherwise be handed an $O(n_\alpha)$ array to discard.

D.9 Truncation threshold

The per-group SVD keeps singular values above $\varepsilon \sigma_{\text{max}}$. Truncating there perturbs the reconstruction at the ε level, so ε is an accuracy knob – but it is only meaningful above the float64 noise floor on singular values, which for an $(n_{\text{row}}, n_{\text{col}})$ matrix is of order $\max(n_{\text{row}}, n_{\text{col}}) \epsilon_{64} \sigma_{\text{max}}$. At these matrices’ sizes that floor can exceed the 10^{-11} which is safe for smaller ones, so the default takes whichever is larger,

$$\varepsilon = \max\left(10^{-11}, 16 \max(n_{\text{row}}, n_{\text{col}}) \epsilon_{64}\right) \quad (\text{D.24})$$

and is computed per group, since the groups differ widely in shape. The CLI exposes it as `varmap df -e`.

D.10 Coarse-graining is a slice, not a maximization

Recall the coarse-grained map of Eq. (1.10): $\bar{A}_{\beta F} = \max_{\alpha \in \beta} A_{\alpha F}$, where $\beta = (d_c, n, p)$ with $0 \leq d_c < 2^{r-L}$ ranges over coarse DM and *subband* rather than multiplet. Computing this by building A and reducing it would defeat the purpose, since building A is the expensive step. With no detrender it is unnecessary:

The group β consists of the fine rows whose subband DM index runs over an *aligned dyadic block*, and by the monotonicity theorem of §B the variance is antitone in the bits of that index. So the maximum over the block is attained at its *bottom*, and Eq. (1.10) is evaluated by a slice rather than a reduction (Eq. B.2 says exactly this).

This is cashed in at the point where a row is written, before any summation over channels and before the SVD. In the level- r labelling the bottom of the block is “all bits below L are zero”, since Eq. (D.18) puts the low bits of the subband DM index into the low L bits of d_{full} ; and since the bit-packing of Eq. (D.13) preserves bit order, those bits occupy the lowest packed positions contiguously. So the slice is a reshape and an index. Taking the *top* of the block instead would produce a min-envelope, which is why the tests check both that some rows were actually sliced and that the resulting map still dominates y^{true} .

The maximization commutes with the group structure per column, which is what licenses doing it row by row: at fixed F every $\alpha \in \beta$ takes its value from one plan entry, since the subbands within a group are disjoint and each subband is assigned to exactly one entry per channel.

The no-detrender hypothesis is load-bearing here. The same appendix shows the monotonicity property is false with a `GpuDetrenderLps2d` in front, so the day a detrender path is added is the day this slice silently becomes wrong.

Coarse-graining is nearly free, and it is the main lever on cost. At `chord_sb2_et.yml` the lifted Q goes 63.8 GiB (fine) $\rightarrow$ 17.5 ($L = R = 4$) $\rightarrow$ 4.3 ($L = 6$) $\rightarrow$ 0.96 ($L = 8$). Almost all of that is the row count: K_{tot} only moves 1435 $\rightarrow$ 1399 at $L = 6$, so what L buys is columns of Q that are 14.6 times shorter, not groups that compress better. The first three passes are unchanged by L by design.

One asymmetry is worth stating because it is easy to assume away: y^{true} of Eq. (1.7) is *fine* whatever the map’s coarse-graining rank, so it cannot be read off the coarse factors. It is accumulated separately, in the full delay-bit basis – which is the second accumulator of §D.7.

D.11 Reducing the rank: global svd-optimization

K_{tot} is the *sum* of the per-group ranks, and nothing in the construction makes columns from different groups independent of each other. So the stacked factorization is routinely far from minimal, and a global pass can reduce it, *exactly*. Given $A = Q \text{ mid } W^T$ at rank K_0 , take thin SVDs $Q = Q'R$ and $W = W'S$ with Q', W' semiorthogonal, so that

$$A = Q'(R \text{ mid } S^T)W'^T \quad (\text{D.25})$$

SVD the $K_0 \times K_0$ middle as $U \text{ diag}(\sigma) V^T$, and keep the K_1 singular values above a numerical cutoff. Then

$$Q_1 = Q'U, \quad \text{mid}_1 = \text{diag}(\sigma), \quad W_1 = W'V \quad (\text{D.26})$$

is a rank- K_1 factorization of the same matrix, with both factors semiorthogonal. If no singular value is below the cutoff the factorization cannot be improved and the map is returned unchanged.

Measured, this removes 24% of the rank at `toy.yml`, 54–57% at the CHIME configs and 59–60% at the CHORD configs, at a relative change in A of 10^{-12} or below – far inside the margin that the admissibility bookkeeping of §1.2 allows, so the `is_admissible` flag survives.

Two implementation notes. The cutoff is *numerical*, not an accuracy knob: it is $K_0 \epsilon_{64}$ relative to the largest singular value, and only removes modes that are numerically absent. (The accuracy knob is ϵ of §D.9, applied per group before this.) And the semiorthogonalization must be done with SVDs, not with the faster Gram-matrix or QR routes: forming $Q^T Q$ squares the condition number, and on these spectra – which

span eleven decades – that was measured to lose seven to eight digits and, worse, to *over-truncate*, reporting a smaller K_1 and discarding real content. A rank-revealing step is exactly where a non-SVD factorization cannot be trusted.

One cost to be aware of: the thin SVD of Q allocates a second array of the same shape, so the peak memory of the map path is about twice the Q of §D.8 when this is on. That is accounted for in the `max_bytes` ceiling, which fails early and says so rather than letting a caller validate a budget and then need double it.

The CLI exposes this as `varmap df -s`, with level 0 (off), 1 (base tree) or 2 (base tree and, after the restriction of §D.12, every other tree).

D.12 The other trees are row restrictions of the base tree

Everything above computes the map of the *base* tree, i.e. the tree with `(primary_tree_index, early_trigger_level) = (0, 0)`. That suffices for the whole config, because with no detrender every other tree’s map is a subset of the base tree’s *rows*, as proved in §A. `compute_detrender_free_multi_map()` caches this in: it computes the base map once and slices it.

For a downsampled primary tree $\gamma > 0$ (Proposition 2, §A.4) the slice keeps the upper half of the coarse-DM axis, every multiplet, and the first P_γ profiles, with W untouched – the restriction acts on rows of A , not columns. Early-trigger trees (Proposition 1) are not stored at all: they are derived from their parent on demand by `VarianceMultiMap`.

Since a row subset of a map that dominates A^{true} elementwise also dominates it, each tree inherits whatever the base map earned, and the `is_admissible` flag carries over.

The restriction can lower the rank: a column of Q whose support lay entirely in the discarded rows is identically zero in the child. Re-running §D.11 on each child recovers that, and measured it is worth a further 0.8% to 3.6%. It is worth knowing why it is small: on the CHIME configs every γ lands on the same rank whatever P_γ is, so what the second pass recovers is the DM-half restriction and not the profile truncation.

Proposition 2 is false with a detrender, and by a wide margin rather than marginally: measured against the brute-force sweep on a small config, agreement is 5×10^{-7} without a detrender and 2.1 with one.

D.13 Av without the map

Production does not need A . What it needs, given the current input-channel variances v_F , is the single vector $y = Av$ per tree, from which the peak-finder’s weights are the inverse square roots. That can be had by skipping the lift of §D.8 entirely: put the weight v_F on the row *sums* – the second accumulator of §D.7 – and lift those. There is no Q , there are no dense group matrices, and there is no SVD.

This is `compute_detrender_free_varfine()`, and the saving is dramatic: at `chord_sb2.et.yml` it delivers all ten trees in 16 seconds within 0.49 GiB, against 63.8 GiB of Q for the base tree alone.

`compute_detrender_free_varcoarse()` is the same result reduced to the granularity the weights array actually has, namely $L = \log_2(\text{wt_dm_downsampling})$ in DM and subbands rather than multiplets in frequency. The reduction is a *max*, because a stored variance has to dominate every output it covers. Note this is not the same thing as coarse-graining the map: $\bar{A}v$ maxes the map and then applies it, and dominates $\max_\alpha (Av)_\alpha$, which is what is computed here.

The two routes share one code path, and share it exactly rather than approximately: `compute_detrender_free_varfine()` for the base tree is *bitwise* equal to the base map’s y^{true} . That is a deliberate property – the weight is applied as a multiplication by exactly 1.0 on a path the two share – and it is asserted by a test, because it is what fails the moment the two paths stop being shared.

This route, and only this route, is ported to C++ (`src_lib/varmap.cpp`, exposed as `pirate_frb.fast_varmap`). The port is 27 times faster at `chord_sb2.et.yml` ($15.1 \text{ s} \rightarrow 0.57 \text{ s}$) and agrees with the python to 1.1×10^{-14} relative. The python remains the reference implementation: the C++ ports only the map-free path, and is tested by asserting that it agrees with the python.

D.14 Validation

Everything here is run by `pirate_frb test --varmap`.

The building blocks are unit-tested against dense reference implementations: the `SparseTile` operations are compared to a dense $\text{DDS}(k)$ applied to randomly generated tiles, tree gridding is compared to `ReferenceTreeGriddingKernel`, and `PfVarianceConvolver` is compared both to a brute-force evaluation of $\|h_p * x\|^2$ and to the kernels which `ReferencePeakFindingKernel` actually uses. The C++ port of each is compared to the python.

End-to-end, the map is checked against the brute-force sweep of §C, which pushes a one-hot through the real dedisperser once per input channel and reads A off the output column norms. The comparison is elementwise over the full matrix, for every primary tree, on random configs. The two sides share no code – one is this appendix’s analysis, the other is the shipped dedisperser – so this is the only numerical oracle for the algorithm, and it is what licenses everything above. Agreement is at the float32 floor of the dedispersion chain, measured 5×10^{-7} to 2×10^{-6} . The same test covers `compute_detrrender_free_varfine()` against the same swept matrices, and covers Proposition 1 of §A by comparing each early-trigger tree’s swept matrix against a row subset of its parent’s.

Several properties which no comparison against another map would catch are tested separately: that the coarse construction of §D.10 agrees with an explicit reduction of the fine map, that the resulting map still dominates y^{true} , that the per-group factorization convention of Eq. (D.22) is not transposed, and that §D.11 recovers the exactly-correct rank on a factorization with known redundancy.

Finally, a stored `VarianceMap` can be checked by Monte Carlo (`pirate_frb varmap mc`): Gaussian noise with per-channel variance V_F is fed through the dedisperser, a `PfSquare` is run over each tree’s subband array, and the resulting empirical per-chunk variances are compared to the map’s prediction, restricted to output channels which have reached steady state. It reports the worst *under*-estimate and the worst *over*-estimate separately, since an admissible map may do the second but never the first.